In [1]:
```
import pandas as pd
import yfinance as yf
from datetime import datetime
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sb
import statsmodels.api as sm
from sklearn import metrics
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import RandomizedSearchCV
from sklearn.svm import SVR
from sklearn.model_selection import GridSearchCV
from sklearn.tree import DecisionTreeRegressor
from sklearn.neural_network import MLPRegressor
from sklearn.datasets import load_iris
from sklearn.neighbors import KNeighborsRegressor
```

In [2]:
```
def get_stock_data(ticker, start_date, end_date):
    # Download stock data
    data = yf.download(ticker, start=start_date, end=end_date)
    return data

# Set the ticker symbol, start date, and end date
ticker = "^NSEI"  # NSE
start_date = "2018-01-01"
end_date = datetime.today().strftime('%Y-%m-%d')

# Get the stock data
nse_data = get_stock_data(ticker, start_date, end_date)
nse_data.dropna(inplace=True)
nse_data
```

```
[**********100%%**********]  1 of 1 completed
```

Out[2]:

Date	Open	High	Low	Close	Adj Close	Volume
2018-01-02	10477.549805	10495.200195	10404.650391	10442.200195	10442.200195	153400
2018-01-03	10482.650391	10503.599609	10429.549805	10443.200195	10443.200195	167300
2018-01-04	10469.400391	10513.000000	10441.450195	10504.799805	10504.799805	174900
2018-01-05	10534.250000	10566.099609	10520.099609	10558.849609	10558.849609	180900
2018-01-08	10591.700195	10631.200195	10588.549805	10623.599609	10623.599609	169000
...	...	...	...	...	...	...
2023-08-14	19383.949219	19465.849609	19257.900391	19434.550781	19434.550781	243900
2023-08-16	19369.000000	19482.750000	19317.199219	19465.000000	19465.000000	0
2023-08-17	19450.550781	19461.550781	19326.250000	19365.250000	19365.250000	268700
2023-08-18	19301.750000	19373.800781	19253.599609	19310.150391	19310.150391	256100
2023-08-21	19320.650391	19425.949219	19296.300781	19393.599609	19393.599609	262600

1389 rows × 6 columns

In [3]:
```
# Calculate Stock High minus Low price (H-L)
nse_data['H-L'] = nse_data['High'] - nse_data['Low']

# Calculate Stock Open minus Close price (O-C)
nse_data['O-C'] = nse_data['Open'] - nse_data['Close']

# Calculate Stock price's seven days' moving average (7 DAYS MA)
nse_data['7 DAYS MA'] = nse_data['Close'].rolling(window=7).mean()

# Calculate Stock price's fourteen days' moving average (14 DAYS MA)
nse_data['14 DAYS MA'] = nse_data['Close'].rolling(window=14).mean()

# Calculate Stock price's twenty one days' moving average (21 DAYS MA)
nse_data['21 DAYS MA'] = nse_data['Close'].rolling(window=21).mean()

# Calculate Stock price's standard deviation for the past seven days (7 DAYS STD DEV)
nse_data['7 DAYS STD DEV'] = nse_data['Close'].rolling(window=7).std()


# Print the data
nse_data.dropna(inplace=True)
```

In [4]:
```
# Calculate descriptive statistics
statistics = nse_data.describe()

# Print the statistics
statistics
```

Out[4]:

	Open	High	Low	Close	Adj Close	Volume	H-L	O-C	7 DAYS M
count	1369.000000	1369.000000	1369.000000	1369.000000	1369.000000	1.369000e+03	1369.000000	1369.000000	1369.0000
mean	13954.957496	14023.454368	13859.526025	13944.029813	13944.029813	4.028783e+05	163.928342	10.927683	13925.8261
std	3175.512607	3180.231113	3167.608887	3176.208944	3176.208944	2.163280e+05	102.007139	117.672030	3164.8283
min	7735.149902	8036.950195	7511.100098	7610.250000	7610.250000	0.000000e+00	26.000000	-847.600586	8264.0428
25%	11044.549805	11081.750000	10964.950195	11047.799805	11047.799805	2.512000e+05	99.299805	-56.599609	11015.9570
50%	12702.150391	12769.750000	12624.849609	12749.150391	12749.150391	3.279000e+05	139.199219	5.299805	12519.4142
75%	17276.000000	17356.250000	17161.150391	17274.300781	17274.300781	5.306000e+05	198.950195	75.449219	17262.7645
max	19850.900391	19991.849609	19758.400391	19979.150391	19979.150391	1.811000e+06	1604.250000	619.650391	19776.8286

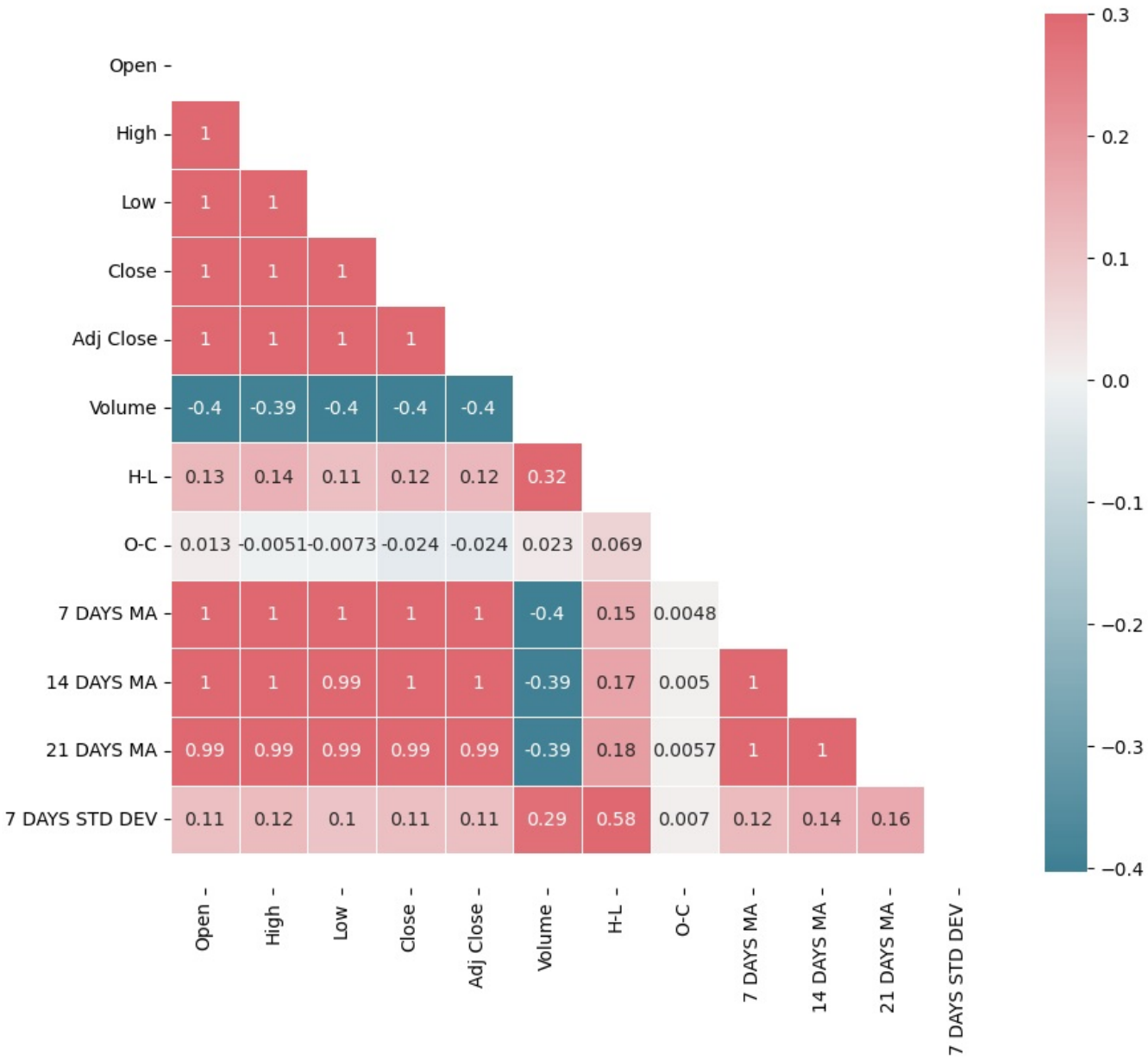
In [5]:
```
# Calculate correlation matrix
correlation_matrix = nse_data.corr()

# Print the correlation matrix
correlation_matrix
```

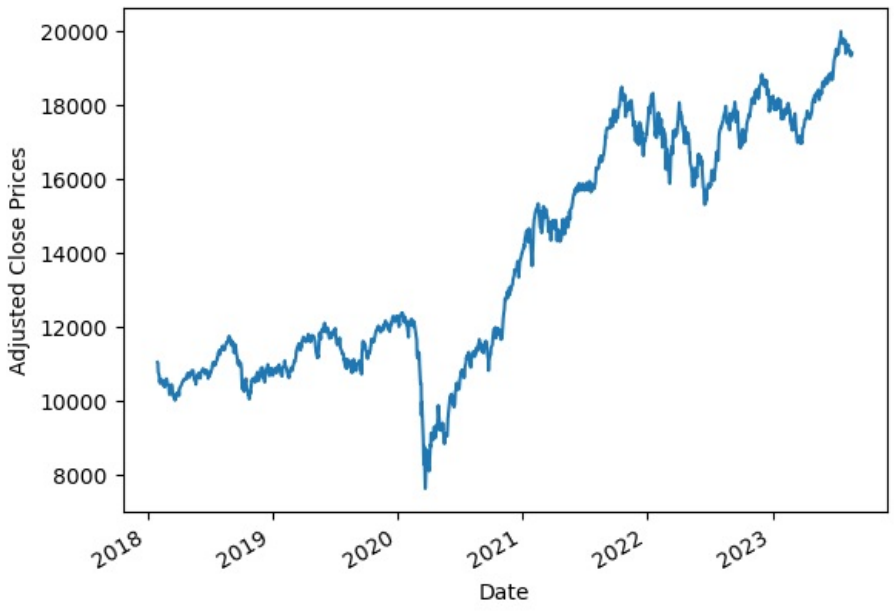
Out[5]:

	Open	High	Low	Close	Adj Close	Volume	H-L	O-C	7 DAYS MA	14 DAYS MA	21 DAYS MA	7 D
Open	1.000000	0.999712	0.999620	0.999314	0.999314	-0.396136	0.126561	0.012610	0.998376	0.995609	0.992734	0.11
High	0.999712	1.000000	0.999491	0.999681	0.999681	-0.391428	0.139531	-0.005069	0.998591	0.996162	0.993528	0.11
Low	0.999620	0.999491	1.000000	0.999672	0.999672	-0.403172	0.107884	-0.007316	0.997779	0.994706	0.991596	0.10
Close	0.999314	0.999681	0.999672	1.000000	1.000000	-0.396884	0.123972	-0.024441	0.997978	0.995205	0.992307	0.10
Adj Close	0.999314	0.999681	0.999672	1.000000	1.000000	-0.396884	0.123972	-0.024441	0.997978	0.995205	0.992307	0.10
Volume	-0.396136	-0.391428	-0.403172	-0.396884	-0.396884	1.000000	0.316274	0.022536	-0.395044	-0.393269	-0.393120	0.28
H-L	0.126561	0.139531	0.107884	0.123972	0.123972	0.316274	1.000000	0.069151	0.148773	0.168489	0.182928	0.58
O-C	0.012610	-0.005069	-0.007316	-0.024441	-0.024441	0.022536	0.069151	1.000000	0.004814	0.005020	0.005653	0.00
7 DAYS MA	0.998376	0.998591	0.997779	0.997978	0.997978	-0.395044	0.148773	0.004814	1.000000	0.998637	0.996259	0.12
14 DAYS MA	0.995609	0.996162	0.994706	0.995205	0.995205	-0.393269	0.168489	0.005020	0.998637	1.000000	0.999061	0.14
21 DAYS MA	0.992734	0.993528	0.991596	0.992307	0.992307	-0.393120	0.182928	0.005653	0.996259	0.999061	1.000000	0.15
7 DAYS STD DEV	0.110179	0.119175	0.100878	0.109896	0.109896	0.285696	0.582922	0.006997	0.121091	0.141488	0.158380	1.00

In [6]:
```
f, ax = plt.subplots(figsize=(10, 12))
corr = nse_data.corr()
mask = np.triu(np.ones_like(corr, dtype=bool))  # Use bool directly
cmap = sb.diverging_palette(220, 10, as_cmap=True)
sb.heatmap(corr, mask=mask, cmap=cmap, vmax=.3, center=0, annot=True,
           square=True, linewidths=.5, cbar_kws={"shrink": .7})
bottom, top = ax.get_ylim()
ax.set_ylim(bottom + 0.5, top - 0.5)
plt.show()
```



In [7]:
```
nse_data['Adj Close'].plot()
plt.ylabel("Adjusted Close Prices")
plt.show()
```



Linear Regression

In [8]:
```
# Select the "Adj Close" and "Date" columns from the stock data
adj_close = nse_data['Adj Close']
high = nse_data['High']
```

```
# Create a DataFrame with "Adj Close" and "Date" columns
data = pd.DataFrame({'Adj Close': adj_close, 'High': high})

# Add a constant term to the independent variable
data = sm.add_constant(data)

# Fit the linear regression model
model = sm.OLS(data['Adj Close'], data[['const', 'High']]).fit()

# Print the regression results
print(model.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:              Adj Close   R-squared:                       0.999
Model:                            OLS   Adj. R-squared:                  0.999
Method:                 Least Squares   F-statistic:                 2.142e+06
Date:                Tue, 22 Aug 2023   Prob (F-statistic):               0.00
Time:                        22:28:54   Log-Likelihood:                -7944.6
No. Observations:                1369   AIC:                         1.589e+04
Df Residuals:                    1367   BIC:                         1.590e+04
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const        -57.2219      9.809     -5.834      0.000     -76.464     -37.980
High           0.9984      0.001   1463.624      0.000       0.997       1.000
==============================================================================
Omnibus:                      637.509   Durbin-Watson:                   1.604
Prob(Omnibus):                  0.000   Jarque-Bera (JB):             3479.895
Skew:                          -2.153   Prob(JB):                         0.00
Kurtosis:                       9.516   Cond. No.                     6.50e+04
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 6.5e+04. This might indicate that there are
strong multicollinearity or other numerical problems.
```

Multiple Regression

In [9]:

```
# Create a DataFrame with the selected columns
data = nse_data[['Open', 'High', 'Low', 'Close', 'Adj Close', 'Volume']]

# Add a constant term to the independent variables
data = sm.add_constant(data)

# Fit the multiple regression model
model = sm.OLS(data['Adj Close'], data[['const', 'Open', 'High', 'Low', 'Close', 'Volume']]).fit()

# Print the regression results
print(model.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:              Adj Close   R-squared:                       1.000
Model:                            OLS   Adj. R-squared:                  1.000
Method:                 Least Squares   F-statistic:                 1.551e+30
Date:                Tue, 22 Aug 2023   Prob (F-statistic):               0.00
Time:                        22:28:54   Log-Likelihood:                 30765.
No. Observations:                1369   AIC:                        -6.152e+04
Df Residuals:                    1363   BIC:                        -6.149e+04
Df Model:                           5
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const       2.046e-12   6.86e-12      0.298      0.765   -1.14e-11    1.55e-11
Open       -4.352e-14   2.55e-14     -1.705      0.089   -9.36e-14    6.57e-15
High       -2.665e-15   2.87e-14     -0.093      0.926   -5.91e-14    5.37e-14
Low         7.105e-15    2.5e-14      0.285      0.776   -4.19e-14    5.61e-14
Close          1.0000   2.59e-14   3.86e+13      0.000       1.000       1.000
Volume       2.32e-17   6.27e-18      3.698      0.000    1.09e-17    3.55e-17
==============================================================================
Omnibus:                       22.641   Durbin-Watson:                   0.032
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               13.230
Skew:                          -0.023   Prob(JB):                      0.00134
Kurtosis:                       2.521   Cond. No.                      2.75e+06
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 2.75e+06. This might indicate that there are
strong multicollinearity or other numerical problems.
```

Random Forest

In [10]:
```
temp = nse_data
temp = temp.drop('Adj Close', axis=1)
x = temp.iloc[:, 0:12].values
y = nse_data.iloc[:, 4].values
```

In [11]:
```
x_train_rf, x_test_rf, y_train_rf, y_test_rf = train_test_split(x, y, test_size=0.20,  random_state=0)
```

In [12]:
```
scale = StandardScaler()
x_train_rf = scale.fit_transform(x_train_rf)
x_test_rf = scale.transform(x_test_rf)
```

In [13]:
```
grid_rf = {
    'n_estimators': [20, 50, 100, 500, 1000],
    'max_depth': np.arange(1, 15, 1),
    'min_samples_split': [2, 10, 9],
    'min_samples_leaf': np.arange(1, 15, 2, dtype=int),
    'bootstrap': [True, False],
    'random_state': [1, 2, 30, 42]
}

# Instantiate the random forest model
model = RandomForestRegressor()

rscv = RandomizedSearchCV(estimator=model, param_distributions=grid_rf, cv=3, n_jobs=-1, verbose=2, n_iter=10, s
rscv_fit = rscv.fit(x_train_rf, y_train_rf)

# Get the best parameters and best model
best_params = rscv_fit.best_params_
best_model = rscv_fit.best_estimator_

print(best_params)
print(best_model)
```

```
Fitting 3 folds for each of 10 candidates, totalling 30 fits
{'random_state': 2, 'n_estimators': 500, 'min_samples_split': 9, 'min_samples_leaf': 1, 'max_depth': 8, 'bootstr
ap': True}
RandomForestRegressor(max_depth=8, min_samples_split=9, n_estimators=500,
                      random_state=2)
```

In [14]:
```
model = RandomForestRegressor(n_estimators=314, random_state=42, min_samples_split=4, min_samples_leaf=3, max_d
model.fit(x_train_rf, y_train_rf)
predict = model.predict(x_test_rf)
print(predict)
```

```
[16699.96980633 14060.44327187 10605.93200734 10125.88339415
 11737.93102955 11031.60191299 11736.02796376 18405.79358548
 11723.35243077 17812.28491944 11311.05961965 18006.08170558
 18600.15360399 11279.75749483 11787.20765017 11691.57418752
 10855.57101875 10731.65873498 10889.43854079 12168.37611677
 17562.00883561 17313.43936359 18343.90632582  9487.6094714
 13852.53113665 16190.42809184 19648.09691393 17556.02401048
 10862.47976463 10365.06460793 17308.17402257 12144.97585418
 15953.67548848 17354.91763865 17391.76361475 10986.02883267
 11172.75507047  9159.64609398 17519.752693   11464.71848485
 17206.28490206 12298.03634617 10883.16352333 10887.91706425
 11923.15755212 15813.18724406 10579.97028817 10576.59924646
 17674.33819807 10034.41429698 18297.89701237 17666.78998513
 10125.14664743 16649.97393729 11988.84145964 11891.75699528
 10684.6664004  12252.52553941 16955.31235984 17273.54131574
 10542.03128852 15165.74244321 10833.6249822  19584.30542696
 17892.51129757 11860.77209418 11057.48890833 17712.77057729
 12181.02351983 10477.40652917 12127.166398   11723.31772914
 10409.92296082 19373.5402678  10618.8895996  17812.42869209
 11754.37521132  9930.38922809 18757.91447418 10576.75818757
 19717.02649322 18159.87630751 16000.65873698 17553.54619178
 19392.6459501  12020.9681734  17054.4323359  16246.74709749
 10946.96588221 10980.62526222 11146.9782972   9188.85676863
 18276.88124018 11710.11572922 16690.7994525  14916.55985602
 17658.23250674 10599.78746228 16606.65278256 10730.76059669
 10476.24122787 16422.93648251 18200.53882592  9268.78750806
 17702.61597121 10791.27941949 19691.36531326 11201.57668697
 10229.72542226 15777.53611846 19450.8346937  16818.26713483
 14645.05905078 11582.98345751 15682.80682506 11502.56091882
 16985.29158307 11520.70562954 10485.12329013 12080.45024211
 17533.61797964 14560.0207896  13907.02189121 17580.95324722
 13074.00273873  8154.40674391 17102.36585966 10968.88337936
 10549.0846815  11273.36212621 10570.77870739 17721.94789176
 10454.75683514  8966.485394   11075.93381849 13831.98623451
 14158.28404202 11250.13899652 17136.56860619 14333.13202569
 10847.14113681 18394.99506092 17075.75475765 11571.67013452
 10211.75919151 15737.76475646 12345.68298169 11020.70302603
  9248.80585104 17918.08952664 15242.47021366 11587.35721235
 15804.07192322 17648.3039737  10692.75340795  9816.46397414
 12031.29600269 17945.4124996  18002.77341006 11647.39564939
 17002.09595488 11843.54829924 10890.6208378  17931.57990146
 18795.31735891 18471.89162466 10739.86099101 17710.97479886
 17557.65045936 10399.04988174 12260.17783902 17764.34185641
 16212.33007822 15835.52029709  9001.19706355 12263.5927884
  9916.03435043 11439.60825562 12261.2929746  11353.93114309
 17202.90218128 11274.38320488 15528.01356726 10494.6364648
 18006.01118004 10807.46395412 18197.43078014 14336.87485294
 18532.75563012 15965.55290009 10619.55903011 17001.52402184
 17502.7661935  10304.88412248 14581.92657836  9038.9355534
 18496.87060835 15744.52167823 18161.35625767 10683.82484195
 16256.61744521 14635.32804856 10818.55409649 13522.28141867
 10073.05988497 10230.14942385 10432.92736119 17043.6805745
 11555.36968448 15530.31380471 17071.43135214  9257.63664409
 19338.10368423 11842.34364212  8167.1354766  16633.11237685
 11306.8453589  10542.85984692 19558.82411639 11278.89617381
 11054.57953325 10579.37272192 10540.74656604  9145.48182332
 17827.99313104 18321.26567624 15860.61603338 11585.47634073
 10430.61319162 16845.43159585 17091.92753893 11516.40536744
 17662.83438293 11992.90910285 17893.8553496  12045.32286613
 11892.33946678 17272.81851783 17623.44637433 12266.55757171
 11913.35330269 11647.49626813 16988.38592994 11842.41896847
 10367.29445205 11871.4819432  10823.01938163 18200.35211849
 11978.12764783  8635.40209241 14691.02047554 11971.1802116
 15678.37313339 12048.9615906  15173.24754535 11552.11171187
 14312.31726401 11306.20477184 11276.92657602 11940.4995362
 18629.05486051 18256.55291177 18322.2542087  17011.52662206
 12840.17519655 17378.05140724 14857.95666146 16955.33158323
 11303.32254657 11660.88284105 11070.25923118 18187.42966354
 17482.65346373 16201.28154304]
```

In [15]:

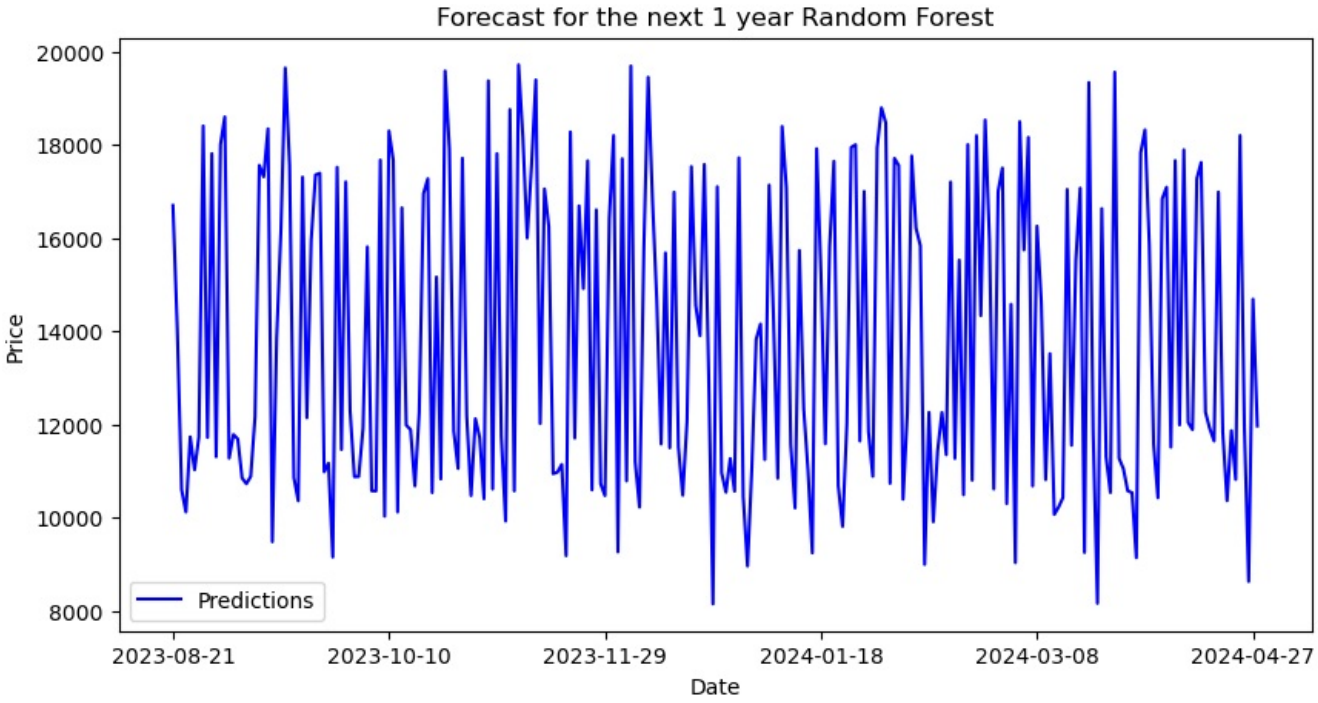
```
print("Mean Absolute Error:", round(metrics.mean_absolute_error(y_test_rf, predict), 4))
print("Mean Squared Error:", round(metrics.mean_squared_error(y_test_rf, predict), 4))
print("Root Mean Squared Error:", round(np.sqrt(metrics.mean_squared_error(y_test_rf, predict)), 4))
print("(R^2) Score:", round(metrics.r2_score(y_test_rf, predict), 4))
print(f'Train Score : {model.score(x_train_rf, y_train_rf) * 100:.2f}% and Test Score : {model.score(x_test_rf,
errors = abs(predict - y_test_rf)
mape = 100 * (errors / y_test_rf)
accuracy = 100 - np.mean(mape)
print('Accuracy:', round(accuracy, 2), '%.')
```

```
Mean Absolute Error: 13.1608
Mean Squared Error: 814.1729
Root Mean Squared Error: 28.5337
(R^2) Score: 0.9999
Train Score : 99.99% and Test Score : 99.99% using Random Tree Regressor.
Accuracy: 99.89 %.
```

In [16]:
```
predictions = pd.DataFrame({"Predictions": predict})
predictions['Date'] = pd.date_range(start=nse_data.index[-1], periods=len(predict), freq="D")
predictions.set_index("Date", inplace=True)
predictions.to_csv("Predicted-price-data_rf_NSE.csv")
#colllects future days from predicted values
oneyear_df = pd.DataFrame(predictions[:252])
oneyear_df.to_csv("one-year-predictions_rf_NSE.csv")
onemonth_df = pd.DataFrame(predictions[:21])
onemonth_df.to_csv("one-month-predictions_rf_NSE.csv")
fivedays_df = pd.DataFrame(predictions[:5])
fivedays_df.to_csv("five-days-predictions_rf_NSE.csv")
```

In [17]:
```
oneyear_df_pred = pd.read_csv("one-year-predictions_rf_NSE.csv")
oneyear_df_pred.set_index("Date", inplace=True)
buy_price = min(oneyear_df_pred["Predictions"])
sell_price = max(oneyear_df_pred["Predictions"])
oneyear_buy = oneyear_df_pred.loc[oneyear_df_pred["Predictions"] == buy_price]
oneyear_sell = oneyear_df_pred.loc[oneyear_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(oneyear_buy)
print("Sell price and date")
print(oneyear_sell)
oneyear_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 1 year Random Forest", color=
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()
```

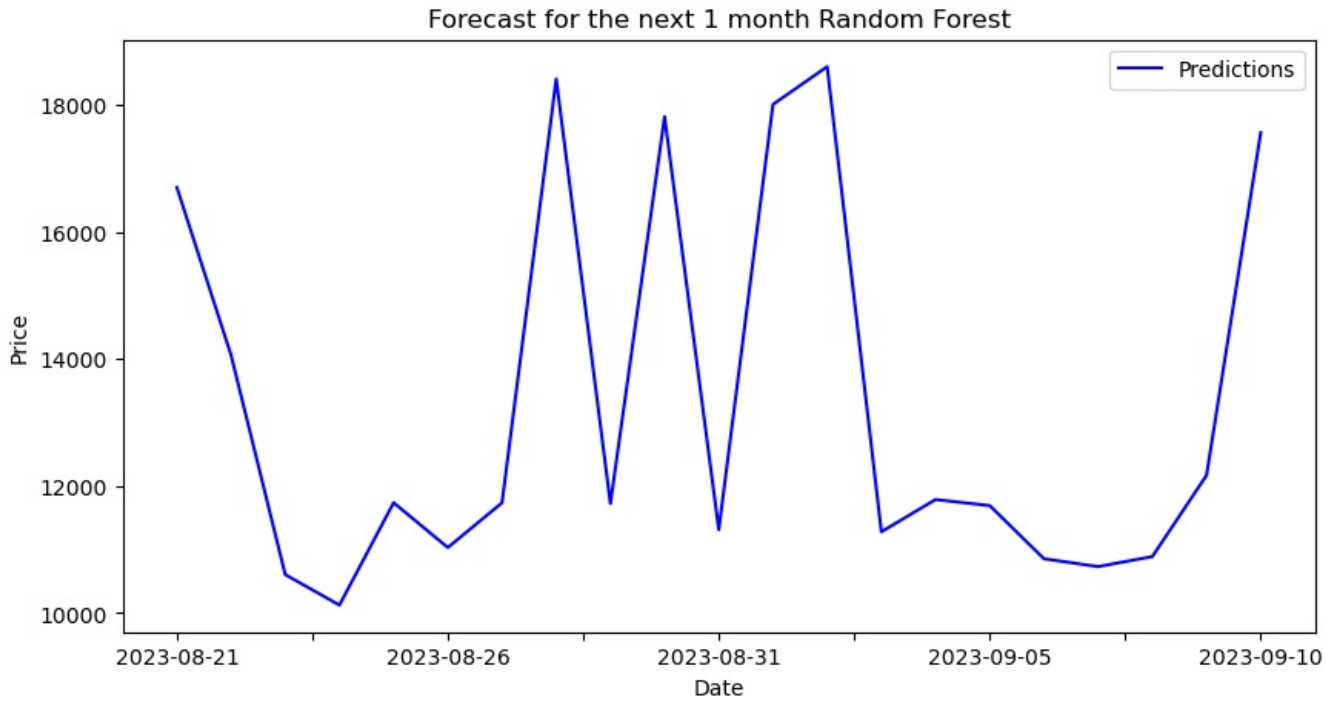
```
Buy price and date
            Predictions
Date
2023-12-24  8154.406744
Sell price and date
             Predictions
Date
2023-11-09  19717.026493
```



In [18]:
```
onemonth_df_pred = pd.read_csv("one-month-predictions_rf_NSE.csv")
onemonth_df_pred.set_index("Date", inplace=True)
buy_price = min(onemonth_df_pred["Predictions"])
sell_price = max(onemonth_df_pred["Predictions"])
onemonth_buy = onemonth_df_pred.loc[onemonth_df_pred["Predictions"] == buy_price]
onemonth_sell = onemonth_df_pred.loc[onemonth_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(onemonth_buy)
print("Sell price and date")
print(onemonth_sell)
onemonth_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 1 month Random Forest", colo
plt.xlabel("Date")
```

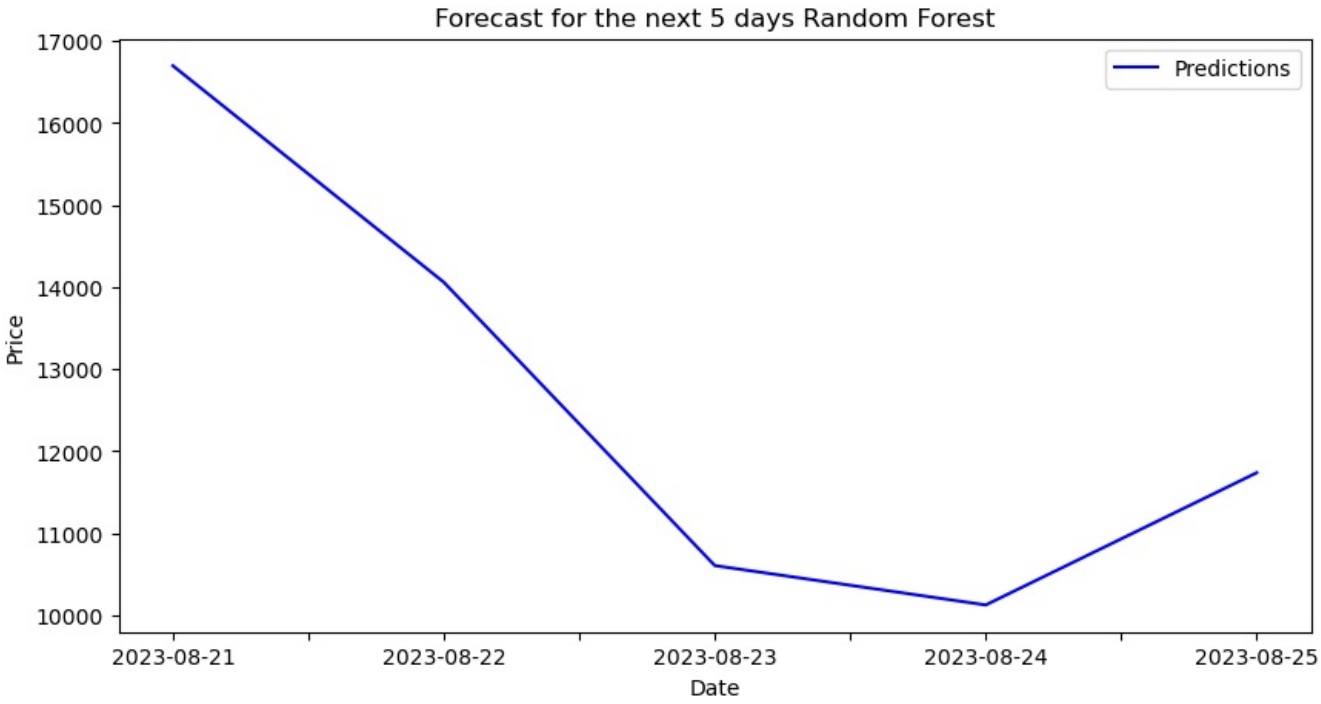
```
plt.ylabel("Price")
plt.legend()
plt.show()
```

```
Buy price and date
            Predictions
Date
2023-08-24  10125.883394
Sell price and date
            Predictions
Date
2023-09-02  18600.153604
```



In [19]:
```
fivedays_df_pred = pd.read_csv("five-days-predictions_rf_NSE.csv")
fivedays_df_pred.set_index("Date", inplace=True)
buy_price = min(fivedays_df_pred["Predictions"])
sell_price = max(fivedays_df_pred["Predictions"])
fivedays_buy = fivedays_df_pred.loc[fivedays_df_pred["Predictions"] == buy_price]
fivedays_sell = fivedays_df_pred.loc[fivedays_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(fivedays_buy)
print("Sell price and date")
print(fivedays_sell)
fivedays_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 5 days Random Forest", color=
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()
```

```
Buy price and date
            Predictions
Date
2023-08-24  10125.883394
Sell price and date
            Predictions
Date
2023-08-21  16699.969806
```



SVM

In [20]:
```
temp = nse_data
temp = temp.drop('Adj Close', axis=1)
x = temp.iloc[:, 0:12].values
y = nse_data.iloc[:, 4].values
```

In [21]:
```
x_train_svm, x_test_svm, y_train_svm, y_test_svm = train_test_split(x, y, test_size=0.20,  random_state=0)
```

In [22]:
```
scale = StandardScaler()
x_train_svm = scale.fit_transform(x_train_svm)
x_test_svm = scale.transform(x_test_svm)
```

In [23]:
```
# Define the parameter grid for hyperparameter tuning
param_grid = {
    'C': [0.1, 1.0, 10.0],
    'gamma': [0.1, 1.0, 'scale'],
    'kernel': ['linear', 'rbf']
}

# Instantiate the SVM model
svm_model = SVR()

# Perform grid search with cross-validation
grid_search = GridSearchCV(estimator=svm_model, param_grid=param_grid, cv=5, scoring='neg_mean_squared_error')
grid_search.fit(x_train_svm, y_train_svm)

# Get the best parameters and best model
best_params = grid_search.best_params_
best_model = grid_search.best_estimator_

print(best_params)
print(best_model)
```

```
{'C': 10.0, 'gamma': 0.1, 'kernel': 'linear'}
SVR(C=10.0, gamma=0.1, kernel='linear')
```

In [24]:
```
# Retrain the best model on the full training set
best_model.fit(x_train_svm, y_train_svm)

# Make predictions
predict = best_model.predict(x_test_svm)
predict
```

Out[24]:
```
array([16682.34159737, 13994.02866572, 10596.69283604, 10112.80546927,
       11728.88078949, 11032.59528188, 11721.28961183, 18449.75404869,
       11731.759228  , 17791.74847995, 11306.27154381, 17978.74598209,
       18607.07033631, 11287.57080142, 11788.31346625, 11683.15940988,
       10843.65531126, 10738.09137109, 10894.215775  , 12214.92247325,
       17537.71627247, 17255.20959731, 18355.01098265,  9336.97292774,
       13913.48517223, 16242.345702  , 19663.9809711 , 17583.62216848,
       10857.95399852, 10348.08559973, 17308.2497878 , 12150.16219573,
       15982.03311974, 17339.32391723, 17329.77845207, 11068.76250342,
       11185.0915923 ,  9177.36145121, 17535.70041481, 11441.27793311,
       17222.26199684, 12282.5753255 , 10860.3756033 , 10885.00747169,
       11901.47180728, 15818.19052693, 10566.9193627 , 10582.45189894,
       17717.26239168, 10015.30584726, 18290.1464547 , 17684.00552887,
       10139.04263957, 16641.16221718, 11999.73409296, 11907.882378  ,
       10688.79102383, 12268.01669405, 16959.38469598, 17244.9288162 ,
       10589.8042279 , 15164.16825516, 10854.29275524, 19620.35939138,
       17922.25070962, 11850.93861849, 11059.89217245, 17736.0895287 ,
       12172.95556015, 10514.60383086, 12160.6390935 , 11737.9848322 ,
       10393.42453679, 19341.22014483, 10615.85136449, 17806.50466109,
       11734.38786186,  9890.76412731, 18754.71262172, 10563.34240756,
       19644.49456798, 18127.17962795, 16049.51432846, 17580.73797496,
       19376.0828034 , 12016.72512812, 17106.80582406, 16352.63921737,
       10963.53880372, 10979.88265595, 11192.09765251,  9281.38518967,
       18309.72188964, 11793.85033609, 16729.98110723, 14924.20702385,
       17691.50263462, 10592.33641021, 16593.33309074, 10731.79197336,
       10581.04680991, 16512.66181231, 18200.48534098,  9286.01261968,
       17672.02005884, 10797.75400149, 19688.66151528, 11187.89120977,
       10225.26643419, 15795.00817918, 19445.66509457, 16847.43111607,
       14656.32434501, 11570.33758147, 15695.74186283, 11493.05418816,
       16975.40858217, 11518.09990303, 10504.34604162, 12078.65758085,
       17552.45580416, 14575.54882894, 14022.49047929, 17627.54508935,
       13080.33318319,  8468.81080977, 17114.11561685, 10982.98917669,
       10540.38440383, 11310.54602272, 10560.04610925, 17720.559855  ,
       10453.03185806,  8855.97709102, 11079.32476605, 13869.07767514,
       14080.3234257 , 11304.63599417, 17204.22547805, 14338.80013316,
       10860.27611302, 18378.96945815, 17090.27056665, 11599.46847339,
       10211.77602044, 15735.44696123, 12360.25475343, 10988.81958359,
        9288.2290238 , 17944.77803239, 15199.51259275, 11589.30654163,
       15795.87799889, 17624.75323989, 10687.27554173,  9817.09480896,
       12030.95484236, 17945.60511862, 17987.69882063, 11670.49192709,
       16974.22311307, 11851.65789854, 10896.21352776, 17957.58453124,
       18737.80256207, 18479.17910513, 10731.35333485, 17644.68082428,
       17522.21093214, 10406.08367674, 12234.46167161, 17758.30245501,
       16174.03529819, 15839.71523577,  8980.04121623, 12254.85144943,
        9997.14261472, 11406.44063597, 12292.22522601, 11336.7673609 ,
       17209.54052233, 11337.90265216, 15387.89243259, 10498.94505395,
       18015.95990269, 10866.47627407, 18202.21868886, 14427.21134746,
       18538.75854466, 15939.77932847, 10625.67469521, 17087.03141883,
       17468.24529638, 10294.81276365, 14573.0362293 ,  9045.74313064,
       18488.16472787, 15717.19908336, 18157.01542351, 10670.12628629,
       16238.47736066, 14625.32508599, 10813.61107246, 13236.03491933,
       10090.15485197, 10244.21834733, 10458.0797518 , 17063.36027279,
       11559.32028192, 15438.11893447, 17109.70700306,  9350.86914504,
       19331.03264359, 11858.35862171,  8096.26575959, 16622.82165769,
       11299.75720416, 10542.98560789, 19552.19150979, 11298.84967745,
       11040.34268684, 10559.61169149, 10564.01825244,  9162.31277882,
       17841.65650708, 18303.02108131, 15851.16910488, 11579.69828402,
       10431.69611512, 16887.13856104, 17147.92074651, 11516.70586243,
       17680.08792381, 11999.03933841, 17891.69558688, 12045.08916934,
       11921.13662247, 17322.67226117, 17652.38368609, 12260.6102026 ,
       11928.65674551, 11655.97184664, 16995.82590671, 11807.18052577,
       10361.68864948, 11874.92296528, 10818.38141466, 18161.45970508,
       11969.77651486,  8579.48039537, 14688.47415553, 11966.91932197,
       15632.25752119, 12048.16480816, 15216.04331681, 11585.36433282,
       14396.32721817, 11354.21326456, 11317.82957584, 11947.92054249,
       18665.68358499, 18266.86234864, 18288.04985699, 17066.39331177,
       12774.05958621, 17351.54153562, 14843.87942876, 16967.61486738,
       11314.07216599, 11634.84509305, 11062.07963589, 18174.028545  ,
       17478.60955468, 16205.98218342])
```

In [25]:
```
print("Mean Absolute Error:", round(metrics.mean_absolute_error(y_test_svm, predict), 4))
print("Mean Squared Error:", round(metrics.mean_squared_error(y_test_svm, predict), 4))
print("Root Mean Squared Error:", round(np.sqrt(metrics.mean_squared_error(y_test_svm, predict)), 4))
print("(R^2) Score:", round(metrics.r2_score(y_test_svm, predict), 4))
```

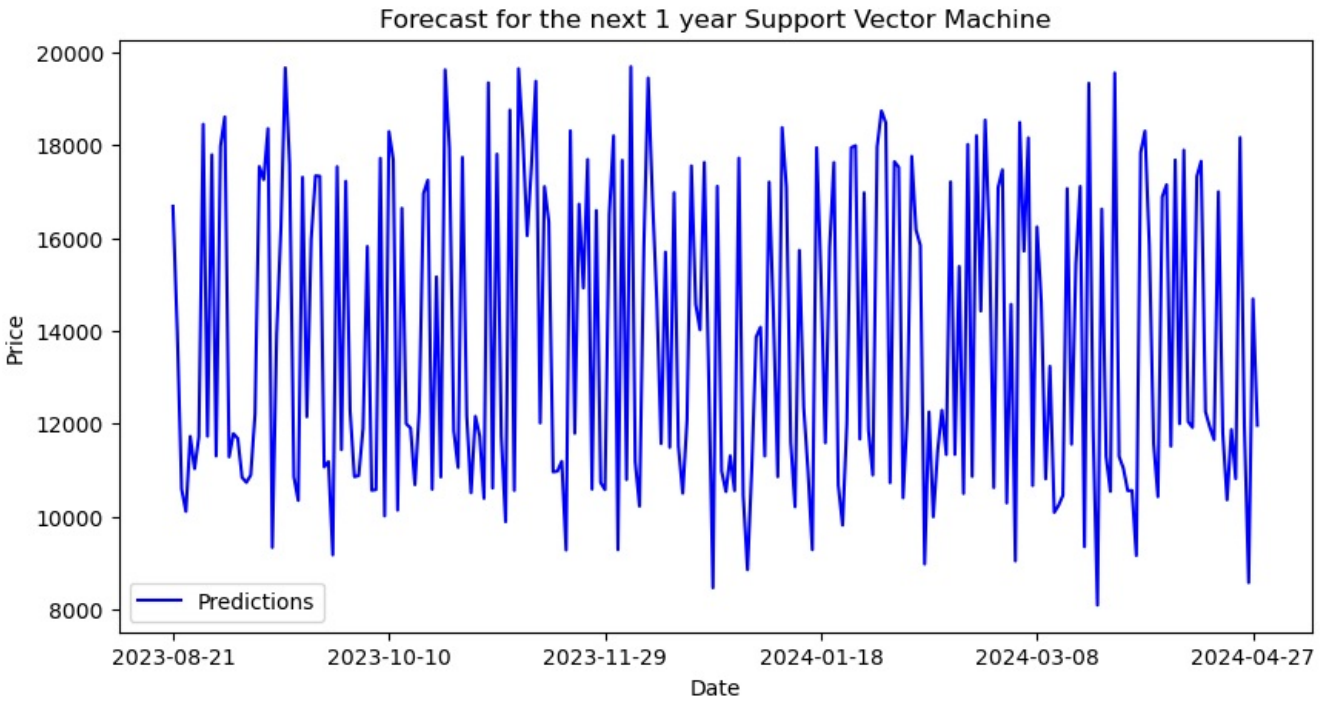
```
print(f'Train Score : {model.score(x_train_svm, y_train_svm) * 100:.2f}% and Test Score : {model.score(x_test_sv
errors = abs(predict - y_test_svm)
mape = 100 * (errors / y_test_svm)
accuracy = 100 - np.mean(mape)
print('Accuracy:', round(accuracy, 2), '%.')
```

```
Mean Absolute Error: 21.7282
Mean Squared Error: 1067.1012
Root Mean Squared Error: 32.6665
(R^2) Score: 0.9999
Train Score : 99.99% and Test Score : 99.99% using Support Vector Regressor.
Accuracy: 99.83 %.
```

In [26]:
```
predictions = pd.DataFrame({"Predictions": predict})
predictions['Date'] = pd.date_range(start=nse_data.index[-1], periods=len(predict), freq="D")
predictions.set_index("Date", inplace=True)
predictions.to_csv("Predicted-price-data_svm_NSE.csv")
#colllects future days from predicted values
oneyear_df = pd.DataFrame(predictions[:252])
oneyear_df.to_csv("one-year-predictions_svm_NSE.csv")
onemonth_df = pd.DataFrame(predictions[:21])
onemonth_df.to_csv("one-month-predictions_svm_NSE.csv")
fivedays_df = pd.DataFrame(predictions[:5])
fivedays_df.to_csv("five-days-predictions_svm_NSE.csv")
```

In [27]:
```
oneyear_df_pred = pd.read_csv("one-year-predictions_svm_NSE.csv")
oneyear_df_pred.set_index("Date", inplace=True)
buy_price = min(oneyear_df_pred["Predictions"])
sell_price = max(oneyear_df_pred["Predictions"])
oneyear_buy = oneyear_df_pred.loc[oneyear_df_pred["Predictions"] == buy_price]
oneyear_sell = oneyear_df_pred.loc[oneyear_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(oneyear_buy)
print("Sell price and date")
print(oneyear_sell)
oneyear_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 1 year Support Vector Machine
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()
```

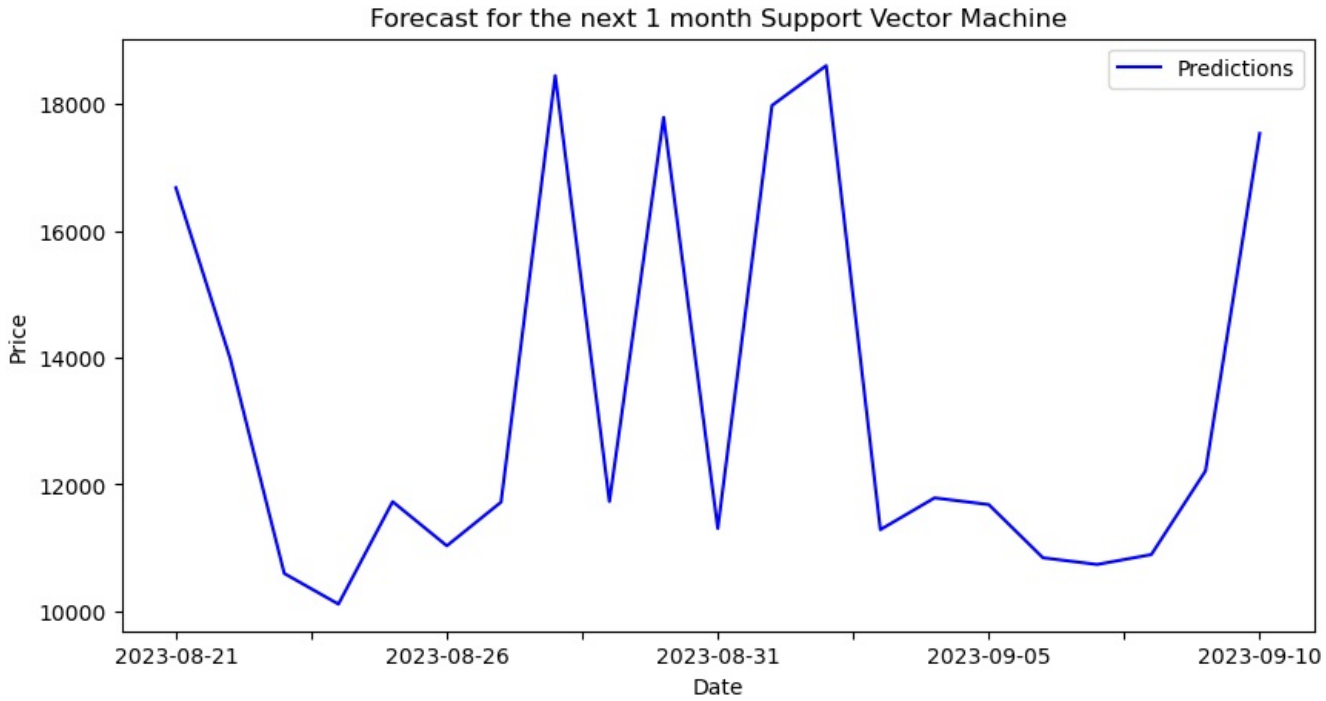
```
Buy price and date
            Predictions
Date
2024-03-22   8096.26576
Sell price and date
             Predictions
Date
2023-12-05  19688.661515
```



In [28]:
```
onemonth_df_pred = pd.read_csv("one-month-predictions_svm_NSE.csv")
onemonth_df_pred.set_index("Date", inplace=True)
buy_price = min(onemonth_df_pred["Predictions"])
sell_price = max(onemonth_df_pred["Predictions"])
onemonth_buy = onemonth_df_pred.loc[onemonth_df_pred["Predictions"] == buy_price]
onemonth_sell = onemonth_df_pred.loc[onemonth_df_pred["Predictions"] == sell_price]
```

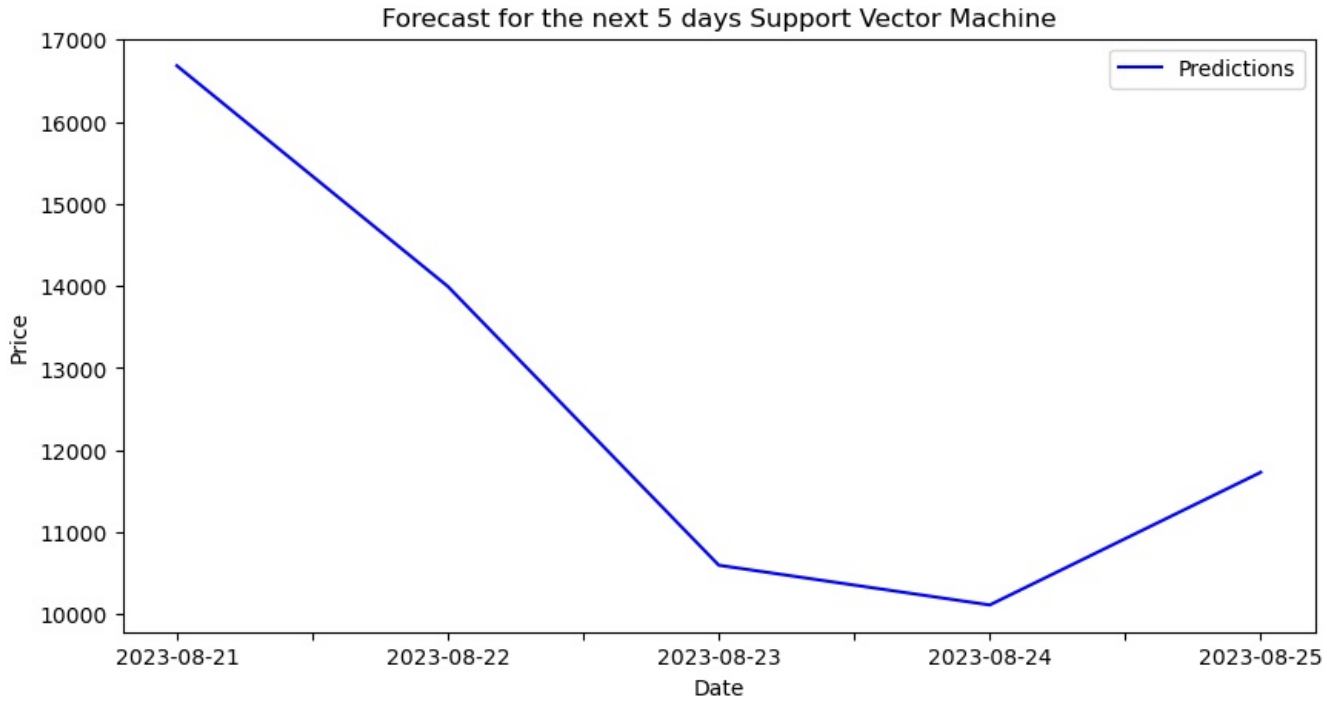
```
print("Buy price and date")
print(onemonth_buy)
print("Sell price and date")
print(onemonth_sell)
onemonth_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 1 month Support Vector Machi
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()
```

```
Buy price and date
            Predictions
Date
2023-08-24  10112.805469
Sell price and date
            Predictions
Date
2023-09-02  18607.070336
```



In [29]:
```
fivedays_df_pred = pd.read_csv("five-days-predictions_svm_NSE.csv")
fivedays_df_pred.set_index("Date", inplace=True)
buy_price = min(fivedays_df_pred["Predictions"])
sell_price = max(fivedays_df_pred["Predictions"])
fivedays_buy = fivedays_df_pred.loc[fivedays_df_pred["Predictions"] == buy_price]
fivedays_sell = fivedays_df_pred.loc[fivedays_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(fivedays_buy)
print("Sell price and date")
print(fivedays_sell)
fivedays_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 5 days Support Vector Machin
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()
```

```
Buy price and date
            Predictions
Date
2023-08-24  10112.805469
Sell price and date
            Predictions
Date
2023-08-21  16682.341597
```



Decision Tree

In [30]:
```
temp = nse_data
temp = temp.drop('Adj Close', axis=1)
x = temp.iloc[:, 0:12].values
y = nse_data.iloc[:, 4].values
```

In [31]:
```
x_train_dt, x_test_dt, y_train_dt, y_test_dt = train_test_split(x, y, test_size=0.20,  random_state=0)
```

In [32]:
```
scale = StandardScaler()
x_train_dt = scale.fit_transform(x_train_dt)
x_test_dt = scale.transform(x_test_dt)
```

In [33]:
```
# Define the parameter grid for hyperparameter tuning
param_grid = {
    'max_depth': [None, 5, 10, 20],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

# Instantiate the Decision Tree model
dt_model = DecisionTreeRegressor()

# Perform grid search with cross-validation
grid_search = GridSearchCV(estimator=dt_model, param_grid=param_grid, cv=5, scoring='neg_mean_squared_error')
grid_search.fit(x_train_dt, y_train_dt)

# Get the best parameters and best model
best_params = grid_search.best_params_
best_model = grid_search.best_estimator_

print(best_params)
print(best_model)
```

```
{'max_depth': 20, 'min_samples_leaf': 1, 'min_samples_split': 2}
DecisionTreeRegressor(max_depth=20)
```

In [34]:
```
# Retrain the best model on the full training set
best_model.fit(x_train_dt, y_train_dt)
```

```
# Make predictions
predict = best_model.predict(x_test_dt)
predict
```

Out[34]:
```
array([16705.19921875, 13981.95019531, 10604.34960938, 10124.90039062,
       11752.79980469, 11023.20019531, 11754.65039062, 18385.30078125,
       11713.20019531, 17812.40039062, 11312.20019531, 18012.19921875,
       18598.65039062, 11278.90039062, 11786.84960938, 11691.90039062,
       10856.70019531, 10724.40039062, 10890.29980469, 12174.65039062,
       17563.94921875, 17311.80078125, 18348.        ,  9380.90039062,
       13981.75      , 16170.15039062, 19680.59960938, 17557.05078125,
       10859.90039062, 10383.        , 17314.65039062, 12137.95019531,
       15924.20019531, 17353.5       , 17392.59960938, 10987.45019531,
       11189.20019531,  9142.75      , 17525.09960938, 11462.20019531,
       17196.69921875, 12329.54980469, 10883.75      , 10883.75      ,
       11930.34960938, 15810.84960938, 10576.29980469, 10576.29980469,
       17674.94921875, 10029.09960938, 18267.25      , 17674.94921875,
       10128.40039062, 16628.        , 11993.04980469, 11896.45019531,
       10685.59960938, 12245.79980469, 16953.94921875, 17274.30078125,
       10545.5       , 15163.29980469, 10831.5       , 19465.        ,
       17898.65039062, 11856.79980469, 11053.90039062, 17711.44921875,
       12165.        , 10488.45019531, 12125.90039062, 11713.20019531,
       10383.        , 19398.5       , 10616.70019531, 17816.25      ,
       11754.65039062,  9914.        , 18755.90039062, 10582.59960938,
       19680.59960938, 18159.94921875, 15966.65039062, 17557.05078125,
       19381.65039062, 12018.40039062, 17063.25      , 16240.29980469,
       10948.25      , 10980.        , 11132.59960938,  9261.84960938,
       18268.40039062, 11724.09960938, 16661.40039062, 14929.5       ,
       17639.55078125, 10596.40039062, 16624.59960938, 10729.84960938,
       10491.04980469, 16416.34960938, 18197.44921875,  9266.75      ,
       17711.44921875, 10791.54980469, 19753.80078125, 11201.75      ,
       10245.25      , 15772.75      , 19465.        , 16842.80078125,
       14634.15039062, 11582.75      , 15691.40039062, 11504.95019531,
       16983.19921875, 11535.        , 10488.45019531, 12088.54980469,
       17532.05078125, 14557.84960938, 13981.95019531, 17576.84960938,
       13109.04980469,  8317.84960938, 17101.94921875, 10961.84960938,
       10545.5       , 11274.20019531, 10582.59960938, 17718.34960938,
       10458.34960938,  8993.84960938, 11101.65039062, 13817.54980469,
       14146.25      , 11244.70019531, 17110.15039062, 14310.79980469,
       10847.90039062, 18385.30078125, 17076.90039062, 11570.90039062,
       10224.75      , 15740.09960938, 12352.34960938, 11017.        ,
        9239.20019531, 17915.05078125, 15293.5       , 11588.34960938,
       15811.84960938, 17656.34960938, 10696.20019531,  9914.        ,
       12025.34960938, 17945.94921875, 17999.19921875, 11641.79980469,
       16983.19921875, 11844.09960938, 10890.29980469, 17925.25      ,
       18771.25      , 18499.34960938, 10739.34960938, 17711.44921875,
       17557.05078125, 10382.70019531, 12245.79980469, 17764.80078125,
       16170.15039062, 15835.34960938,  9039.25      , 12255.84960938,
        9998.04980469, 11435.09960938, 12245.79980469, 11355.75      ,
       17200.80078125, 11274.20019531, 15638.79980469, 10488.45019531,
       17991.94921875, 10806.65039062, 18197.44921875, 14341.34960938,
       18512.75      , 15966.65039062, 10616.70019531, 16983.19921875,
       17511.25      , 10303.54980469, 14563.45019531,  8993.84960938,
       18496.59960938, 15752.40039062, 18159.94921875, 10682.20019531,
       16258.25      , 14617.84960938, 10813.45019531, 13634.59960938,
       10116.15039062, 10249.25      , 10421.40039062, 17053.94921875,
       11551.75      , 15556.65039062, 17053.94921875,  9205.59960938,
       19355.90039062, 11844.09960938,  8317.84960938, 16614.19921875,
       11300.54980469, 10551.70019531, 19517.        , 11278.90039062,
       11053.90039062, 10585.20019531, 10536.70019531,  9199.04980469,
       17828.        , 18321.15039062, 15856.04980469, 11589.09960938,
       10421.40039062, 16842.80078125, 17117.59960938, 11515.20019531,
       17639.55078125, 11999.09960938, 17898.65039062, 12055.79980469,
       11877.45019531, 17266.75      , 17624.05078125, 12262.75      ,
       11914.40039062, 11641.79980469, 17014.34960938, 11844.09960938,
       10360.40039062, 11872.09960938, 10813.45019531, 18197.44921875,
       11962.09960938,  8641.45019531, 14677.79980469, 11962.09960938,
       15670.25      , 12055.79980469, 15208.45019531, 11558.59960938,
       14341.34960938, 11303.29980469, 11278.        , 11946.75      ,
       18634.55078125, 18268.40039062, 18321.15039062, 17016.30078125,
       12859.04980469, 17382.        , 14867.34960938, 16953.94921875,
       11300.54980469, 11661.84960938, 11067.45019531, 18178.09960938,
       17450.90039062, 16170.15039062])
```

In [35]:
```
print("Mean Absolute Error:", round(metrics.mean_absolute_error(y_test_dt, predict), 4))
print("Mean Squared Error:", round(metrics.mean_squared_error(y_test_dt, predict), 4))
print("Root Mean Squared Error:", round(np.sqrt(metrics.mean_squared_error(y_test_dt, predict)), 4))
print("(R^2) Score:", round(metrics.r2_score(y_test_dt, predict), 4))
print(f'Train Score : {model.score(x_train_dt, y_train_dt) * 100:.2f}% and Test Score : {model.score(x_test_dt,
errors = abs(predict - y_test_dt)
mape = 100 * (errors / y_test_dt)
accuracy = 100 - np.mean(mape)
```

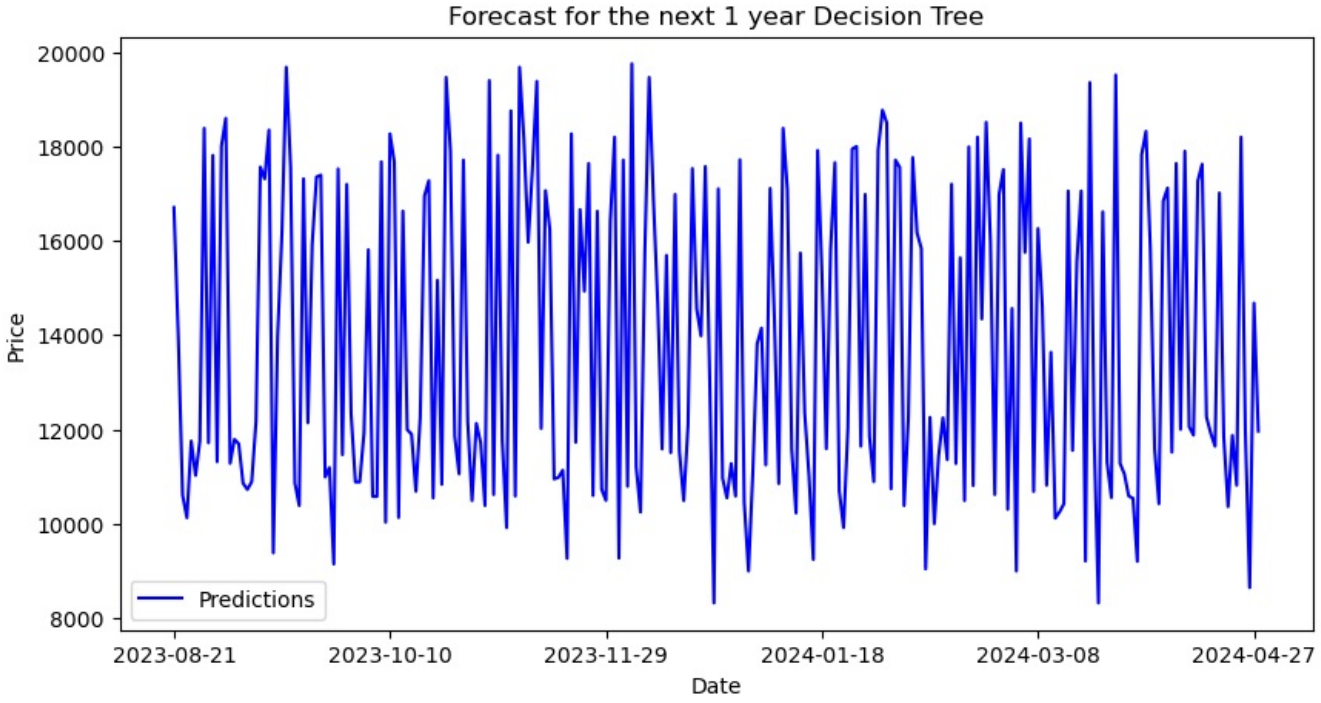
```
print('Accuracy:', round(accuracy, 2), '%.')
```

```
Mean Absolute Error: 16.7
Mean Squared Error: 1417.9829
Root Mean Squared Error: 37.6561
(R^2) Score: 0.9999
Train Score : 99.99% and Test Score : 99.99% using Decision Tree Regressor.
Accuracy: 99.87 %.
```

In [36]:
```
predictions = pd.DataFrame({"Predictions": predict})
predictions['Date'] = pd.date_range(start=nse_data.index[-1], periods=len(predict), freq="D")
predictions.set_index("Date", inplace=True)
predictions.to_csv("Predicted-price-data_dt_NSE.csv")
#colllects future days from predicted values
oneyear_df = pd.DataFrame(predictions[:252])
oneyear_df.to_csv("one-year-predictions_dt_NSE.csv")
onemonth_df = pd.DataFrame(predictions[:21])
onemonth_df.to_csv("one-month-predictions_dt_NSE.csv")
fivedays_df = pd.DataFrame(predictions[:5])
fivedays_df.to_csv("five-days-predictions_dt_NSE.csv")
```

In [37]:
```
oneyear_df_pred = pd.read_csv("one-year-predictions_dt_NSE.csv")
oneyear_df_pred.set_index("Date", inplace=True)
buy_price = min(oneyear_df_pred["Predictions"])
sell_price = max(oneyear_df_pred["Predictions"])
oneyear_buy = oneyear_df_pred.loc[oneyear_df_pred["Predictions"] == buy_price]
oneyear_sell = oneyear_df_pred.loc[oneyear_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(oneyear_buy)
print("Sell price and date")
print(oneyear_sell)
oneyear_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 1 year Decision Tree", color=
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()
```

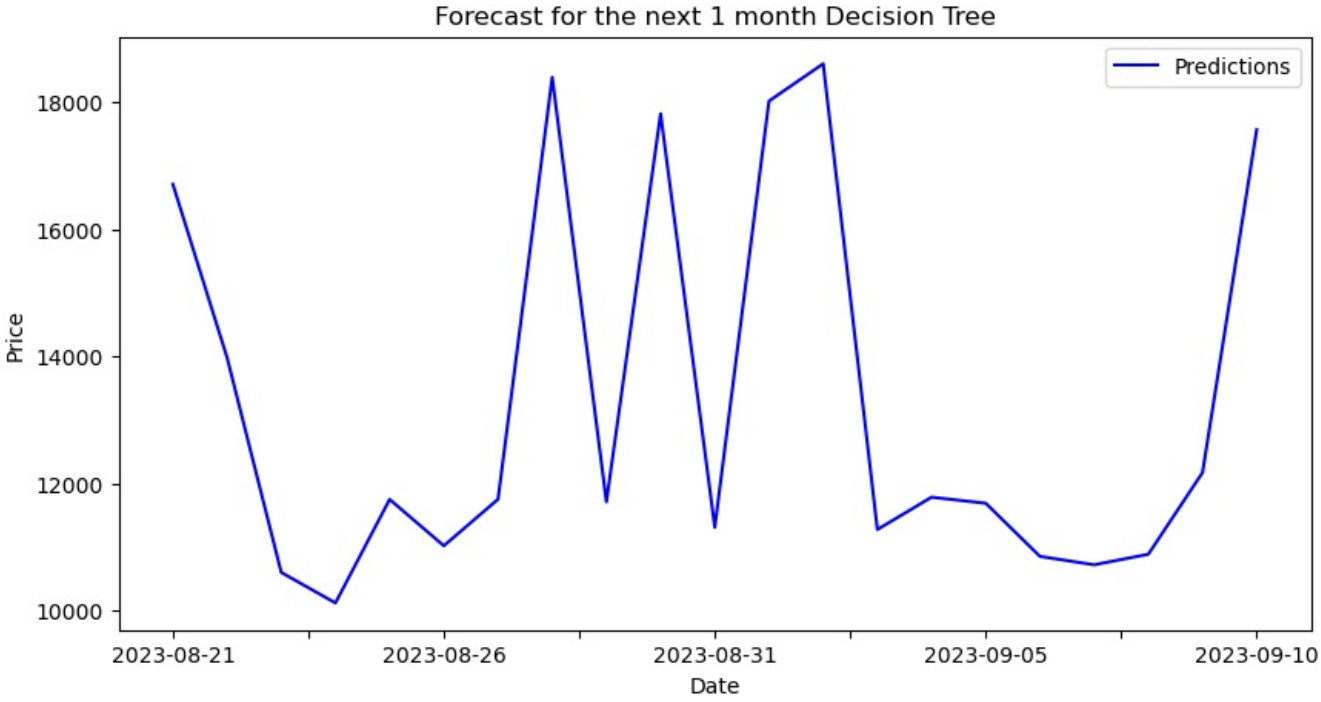
```
Buy price and date
            Predictions
Date
2023-12-24  8317.849609
2024-03-22  8317.849609
Sell price and date
             Predictions
Date
2023-12-05  19753.800781
```



In [38]:
```
onemonth_df_pred = pd.read_csv("one-month-predictions_dt_NSE.csv")
onemonth_df_pred.set_index("Date", inplace=True)
buy_price = min(onemonth_df_pred["Predictions"])
sell_price = max(onemonth_df_pred["Predictions"])
onemonth_buy = onemonth_df_pred.loc[onemonth_df_pred["Predictions"] == buy_price]
onemonth_sell = onemonth_df_pred.loc[onemonth_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(onemonth_buy)
print("Sell price and date")
```

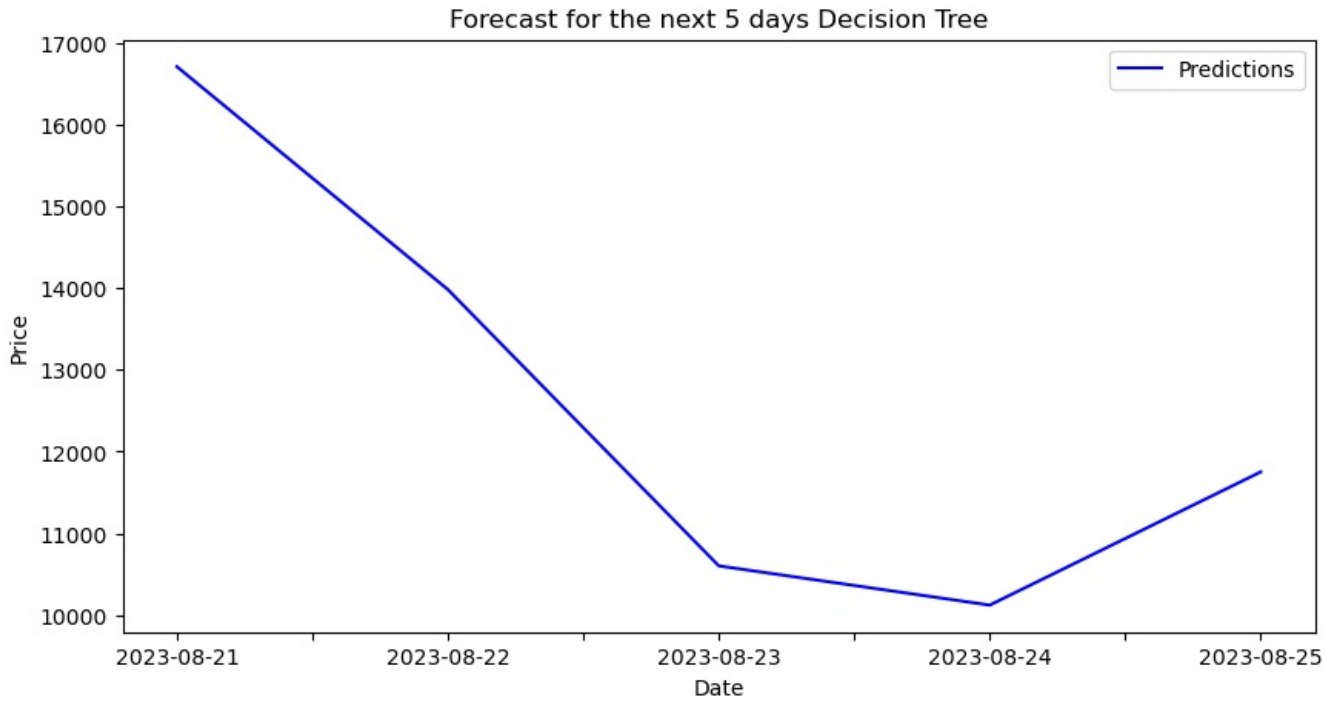
```
print(onemonth_sell)
onemonth_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 1 month Decision Tree", colo
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()
```

```
Buy price and date
            Predictions
Date
2023-08-24  10124.900391
Sell price and date
            Predictions
Date
2023-09-02  18598.650391
```



In [39]:
```
fivedays_df_pred = pd.read_csv("five-days-predictions_dt_NSE.csv")
fivedays_df_pred.set_index("Date", inplace=True)
buy_price = min(fivedays_df_pred["Predictions"])
sell_price = max(fivedays_df_pred["Predictions"])
fivedays_buy = fivedays_df_pred.loc[fivedays_df_pred["Predictions"] == buy_price]
fivedays_sell = fivedays_df_pred.loc[fivedays_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(fivedays_buy)
print("Sell price and date")
print(fivedays_sell)
fivedays_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 5 days Decision Tree", color
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()
```

```
Buy price and date
            Predictions
Date
2023-08-24  10124.900391
Sell price and date
            Predictions
Date
2023-08-21  16705.199219
```



Logistic Regression - ValueError: Unknown label type: 'continuous'

Multilayer Perceptron

In [40]:
```
temp = nse_data
temp = temp.drop('Adj Close', axis=1)
x = temp.iloc[:, 0:12].values
y = nse_data.iloc[:, 4].values
```

In [41]:
```
x_train_mlp, x_test_mlp, y_train_mlp, y_test_mlp = train_test_split(x, y, test_size=0.20,  random_state=0)
```

In [42]:
```
scale = StandardScaler()
x_train_mlp = scale.fit_transform(x_train_mlp)
x_test_mlp = scale.transform(x_test_mlp)
```

In [43]:
```
# Define the parameter grid to search over
param_grid = {
    'hidden_layer_sizes': [(100,), (50, 50), (20, 20, 20)],
    'activation': ['relu', 'tanh'],
    'alpha': [0.0001, 0.001, 0.01],
    'learning_rate': ['constant', 'adaptive'],
}

# Create the MLP regressor
mlp = MLPRegressor(max_iter=1000)

rscv_mlp = RandomizedSearchCV(estimator=mlp, param_distributions=param_grid, cv=3, n_jobs=-1, verbose=2, n_iter=
rscv_fit_mlp = rscv_mlp.fit(x_train_mlp, y_train_mlp)

# Get the best parameters and best model
best_params = rscv_fit_mlp.best_params_
best_model = rscv_fit_mlp.best_estimator_

print(best_params)
print(best_model)
```

```
Fitting 3 folds for each of 10 candidates, totalling 30 fits
{'learning_rate': 'adaptive', 'hidden_layer_sizes': (20, 20, 20), 'alpha': 0.001, 'activation': 'relu'}
MLPRegressor(alpha=0.001, hidden_layer_sizes=(20, 20, 20),
             learning_rate='adaptive', max_iter=1000)
```

```
C:\Users\2227557\AppData\Local\anaconda3\Lib\site-packages\sklearn\neural_network\_multilayer_perceptron.py:691:
ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converge
d yet.
  warnings.warn(
```

In [44]:

```
# Retrain the best model on the full training set
best_model.fit(x_train_mlp, y_train_mlp)

# Make predictions
predict = best_model.predict(x_test_mlp)
predict
```

```
C:\Users\2227557\AppData\Local\anaconda3\Lib\site-packages\sklearn\neural_network\_multilayer_perceptron.py:691:
ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converge
d yet.
  warnings.warn(
```

Out[44]:
```
array([16672.87281456, 13851.58073463, 10608.50490495, 10162.90692509,
       11682.64085277, 11044.13092   , 11665.31236076, 18464.82318356,
       11696.88466577, 17774.81922771, 11314.64936634, 18021.10637435,
       18673.60296696, 11525.53948633, 11767.70356787, 11650.66100478,
       10853.08971938, 10775.88040314, 10903.69462128, 12274.11171579,
       17541.9758085 , 17333.12628133, 18385.04522791,  9637.78041931,
       13908.83829035, 16207.48037204, 19709.24285   , 17562.68778046,
       10850.35908519, 10340.69273585, 17235.68622739, 12180.66130012,
       15982.83466716, 17379.3262114 , 17326.74667618, 11736.41612189,
       11179.64808255,  9290.05257713, 17530.37028307, 11394.04534951,
       17137.28170529, 12342.43727428, 10851.51052862, 10911.61698984,
       11863.70212828, 15840.00870093, 10612.35495516, 10629.35218282,
       17741.93292284, 10042.37645016, 18307.61636783, 17709.05643133,
       10165.28287515, 16663.22559128, 11995.12359869, 11900.80930399,
       10710.79652847, 12277.73093555, 16889.77199865, 17252.32684512,
       10610.39725358, 15225.27922163, 10850.1862606 , 19672.16117137,
       17916.19005564, 11792.75318105, 10931.56194284, 17790.08071312,
       12159.91872728, 10395.43720054, 12168.25038852, 11695.67779393,
       10451.34444494, 19349.62084987, 10669.05663967, 17840.37483013,
       11722.14811554, 10010.60377658, 18784.04440757, 10652.99878687,
       19742.5297357 , 18109.06666327, 16055.18716625, 17643.91283561,
       19399.55915611, 11961.89751248, 17031.88645695, 16287.92763719,
       10991.14388939, 10934.98340307, 11048.74752549,  9383.7919281 ,
       18322.20307602, 11805.80331567, 16708.12908915, 14943.48013849,
       17672.84690568, 10633.76213481, 16620.67135706, 10725.01905597,
       10371.14860846, 16504.40173582, 18223.18802632,  9438.17350322,
       17721.10993996, 10803.94582626, 19737.41096759, 11145.62290113,
       10274.27218369, 15761.08309702, 19431.7839035 , 16777.67679974,
       14653.58078879, 11545.84183734, 15683.30836359, 11439.34296864,
       16887.39418751, 11504.47457742, 10517.22220892, 12076.68266104,
       17564.2576668 , 14547.30204518, 14046.82716493, 17658.69410787,
       13094.99210139,  8541.43319942, 17091.42286706, 11142.62087023,
       10550.80069745, 11233.48142192, 10635.22595532, 17700.66639371,
       10509.60547217,  8721.79216308, 11075.10978498, 13890.21475453,
       14060.64319666, 11257.73567476, 17194.86531433, 14184.75297348,
       10882.60228104, 18410.42517502, 17081.87999996, 11525.64901384,
       10260.72929257, 15770.93159333, 12351.09891135, 10890.56641055,
        9381.41664677, 17958.62889088, 15207.34074393, 11572.04121462,
       15834.26286611, 17625.81397801, 10719.33477008,  9869.51721379,
       11983.61822175, 17939.35822836, 17961.38519912, 11645.33176577,
       16943.18945914, 11823.42274684, 10854.01244045, 17949.72339094,
       18711.93202946, 18482.56810535, 10751.85296802, 17655.85299477,
       17523.46899259, 10436.439273  , 12213.81161446, 17747.18579404,
       16155.50112852, 15857.45969051,  9153.42951819, 12276.94535645,
       10381.17044778, 11321.90255449, 12297.82223753, 11318.39301768,
       17224.30599881, 11359.77496756, 15803.5248301 , 10510.53110384,
       18119.96735736, 10745.45791477, 18187.85722899, 14400.45829331,
       18586.56118937, 15877.87243603, 10634.62785731, 17016.61911135,
       17526.93375499, 10353.64400807, 14602.71340384,  9094.16591524,
       18499.34056703, 15656.3639062 , 18177.00693301, 10717.84745485,
       16261.33467527, 14636.41355832, 10841.37586046, 13658.22184066,
       10152.66347567, 10296.24881275, 10447.80742932, 17032.23424993,
       11494.11657667, 15320.59514127, 17123.42918828,  9083.2090999 ,
       19331.40354799, 11858.10678297,  8588.93207425, 16565.89610834,
       11275.26868335, 10573.0348731 , 19573.68691424, 11185.37955954,
       11053.68441493, 10664.78188323, 10572.01800379,  9329.07648229,
       17835.84902517, 18380.00351722, 15827.75189238, 11593.21283796,
       10447.96252312, 16775.4029188 , 17169.90112511, 11414.0191989 ,
       17656.83268044, 12000.68947422, 17924.96953525, 12017.93351543,
       11921.84045803, 17302.94620063, 17646.56684982, 12258.91343434,
       11954.9555075 , 11578.63887885, 17088.94115724, 11744.49055213,
       10390.26657858, 11890.19078913, 10845.01116465, 18168.40238368,
       11969.40005855,  8334.66758987, 14604.40795433, 11932.71055086,
       15570.55100824, 12043.69463814, 15253.74327079, 11516.46494248,
       14385.41101975, 11304.65307615, 11300.49071343, 11968.04143462,
       18741.7433376 , 18312.3009524 , 18325.45491823, 17172.49048651,
       12747.12953148, 17377.253077  , 14826.46311428, 16930.69668778,
       11291.48541375, 11613.14054603, 11042.05655953, 18172.12747218,
       17500.81211506, 16217.27960544])
```

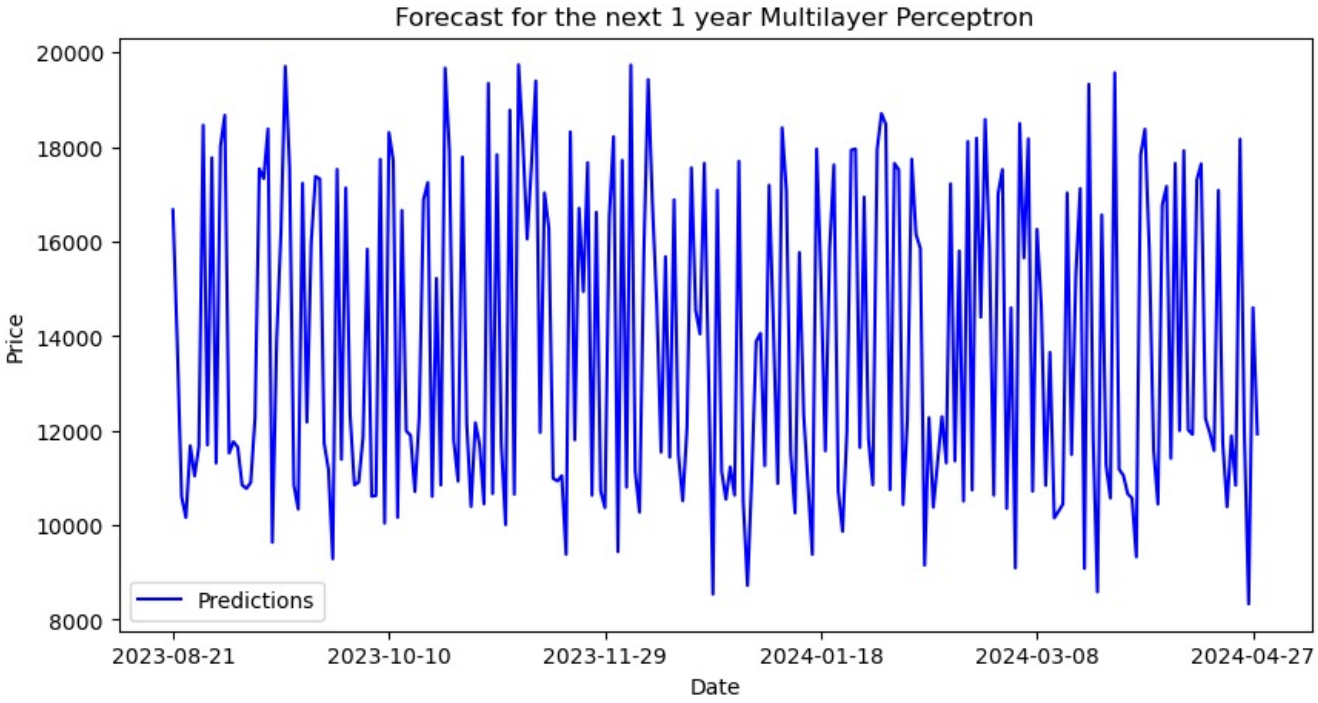
In [45]:
```
print("Mean Absolute Error:", round(metrics.mean_absolute_error(y_test_mlp, predict), 4))
print("Mean Squared Error:", round(metrics.mean_squared_error(y_test_mlp, predict), 4))
print("Root Mean Squared Error:", round(np.sqrt(metrics.mean_squared_error(y_test_mlp, predict)), 4))
print("(R^2) Score:", round(metrics.r2_score(y_test_mlp, predict), 4))
print(f'Train Score : {model.score(x_train_mlp, y_train_mlp) * 100:.2f}% and Test Score : {model.score(x_test_m
errors = abs(predict - y_test_mlp)
mape = 100 * (errors / y_test_mlp)
accuracy = 100 - np.mean(mape)
print('Accuracy:', round(accuracy, 2), '%.')
```

```
Mean Absolute Error: 52.8835
Mean Squared Error: 8733.1844
Root Mean Squared Error: 93.4515
(R^2) Score: 0.9992
Train Score : 99.99% and Test Score : 99.99% using Multilayer Perceptron Regressor.
Accuracy: 99.56 %.
```

In [46]:
```
predictions = pd.DataFrame({"Predictions": predict})
predictions['Date'] = pd.date_range(start=nse_data.index[-1], periods=len(predict), freq="D")
predictions.set_index("Date", inplace=True)
predictions.to_csv("Predicted-price-data_mlp_NSE.csv")
#colllects future days from predicted values
oneyear_df = pd.DataFrame(predictions[:252])
oneyear_df.to_csv("one-year-predictions_mlp_NSE.csv")
onemonth_df = pd.DataFrame(predictions[:21])
onemonth_df.to_csv("one-month-predictions_mlp_NSE.csv")
fivedays_df = pd.DataFrame(predictions[:5])
fivedays_df.to_csv("five-days-predictions_mlp_NSE.csv")
```

In [47]:
```
oneyear_df_pred = pd.read_csv("one-year-predictions_mlp_NSE.csv")
oneyear_df_pred.set_index("Date", inplace=True)
buy_price = min(oneyear_df_pred["Predictions"])
sell_price = max(oneyear_df_pred["Predictions"])
oneyear_buy = oneyear_df_pred.loc[oneyear_df_pred["Predictions"] == buy_price]
oneyear_sell = oneyear_df_pred.loc[oneyear_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(oneyear_buy)
print("Sell price and date")
print(oneyear_sell)
oneyear_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 1 year Multilayer Perceptron"
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()
```

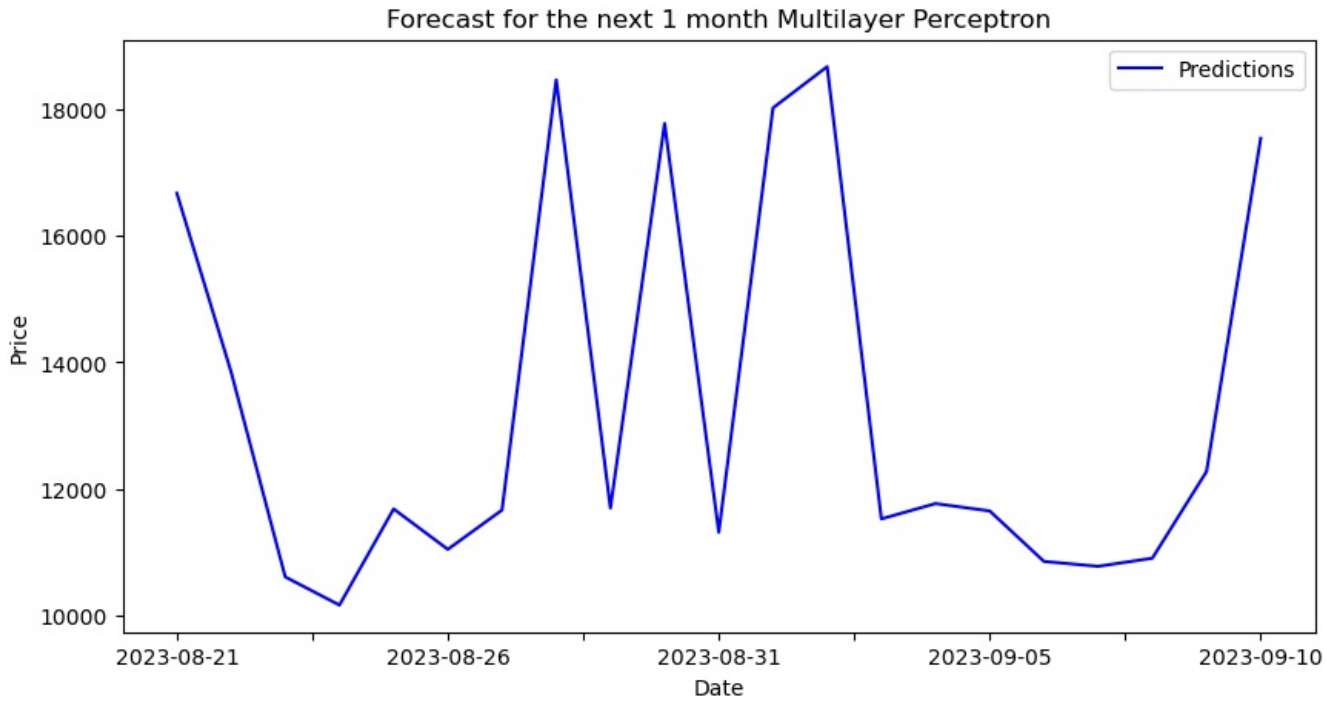
```
Buy price and date
            Predictions
Date
2024-04-26   8334.66759
Sell price and date
             Predictions
Date
2023-11-09  19742.529736
```



In [48]:
```
onemonth_df_pred = pd.read_csv("one-month-predictions_mlp_NSE.csv")
onemonth_df_pred.set_index("Date", inplace=True)
buy_price = min(onemonth_df_pred["Predictions"])
sell_price = max(onemonth_df_pred["Predictions"])
onemonth_buy = onemonth_df_pred.loc[onemonth_df_pred["Predictions"] == buy_price]
onemonth_sell = onemonth_df_pred.loc[onemonth_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(onemonth_buy)
print("Sell price and date")
print(onemonth_sell)
onemonth_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 1 month Multilayer Perceptro
plt.xlabel("Date")
```

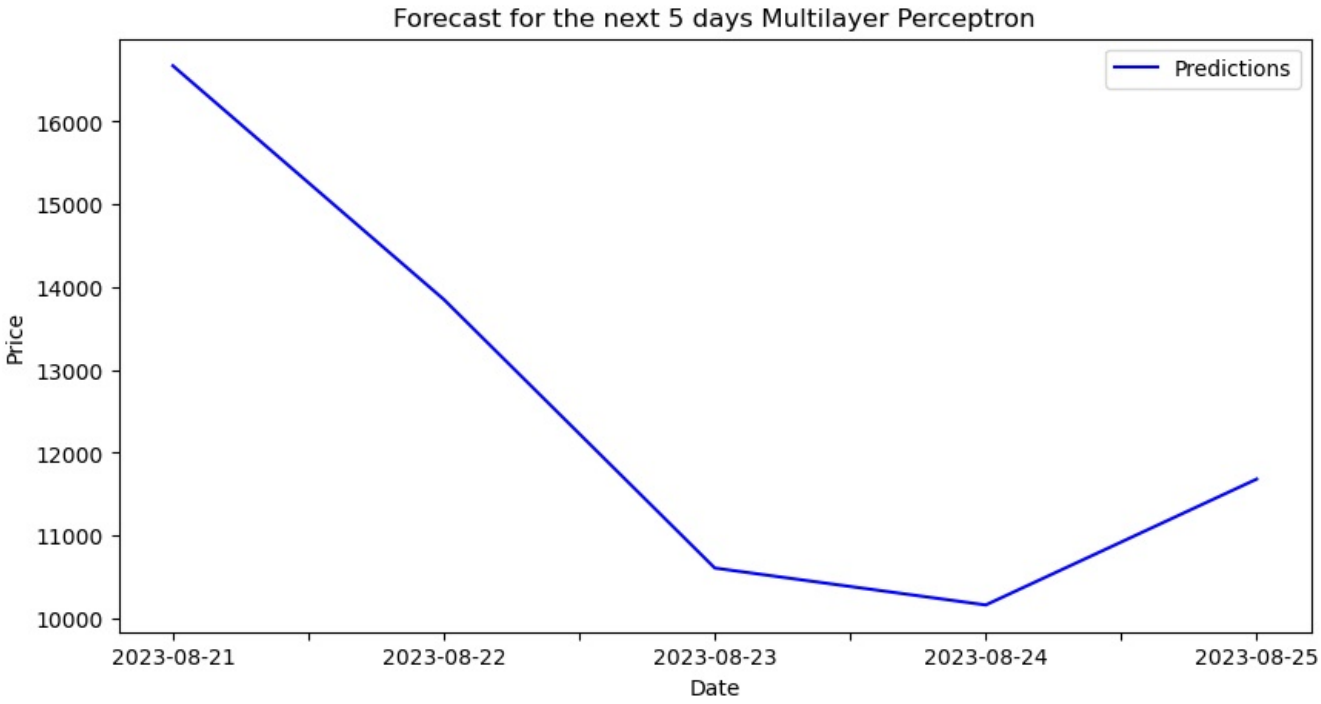
```
plt.ylabel("Price")
plt.legend()
plt.show()
```

```
Buy price and date
            Predictions
Date
2023-08-24  10162.906925
Sell price and date
            Predictions
Date
2023-09-02  18673.602967
```



In [49]:
```
fivedays_df_pred = pd.read_csv("five-days-predictions_mlp_NSE.csv")
fivedays_df_pred.set_index("Date", inplace=True)
buy_price = min(fivedays_df_pred["Predictions"])
sell_price = max(fivedays_df_pred["Predictions"])
fivedays_buy = fivedays_df_pred.loc[fivedays_df_pred["Predictions"] == buy_price]
fivedays_sell = fivedays_df_pred.loc[fivedays_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(fivedays_buy)
print("Sell price and date")
print(fivedays_sell)
fivedays_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 5 days Multilayer Perceptron
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()
```

```
Buy price and date
            Predictions
Date
2023-08-24  10162.906925
Sell price and date
            Predictions
Date
2023-08-21  16672.872815
```



KNN

In [50]:
```
temp = nse_data
temp = temp.drop('Adj Close', axis=1)
x = temp.iloc[:, 0:12].values
y = nse_data.iloc[:, 4].values
```

In [51]:
```
x_train_knn, x_test_knn, y_train_knn, y_test_knn = train_test_split(x, y, test_size=0.20,  random_state=0)
```

In [52]:
```
scale = StandardScaler()
x_train_knn = scale.fit_transform(x_train_knn)
x_test_knn = scale.transform(x_test_knn)
```

In [53]:
```
# Define the parameter grid to search over
param_grid = {
    'n_neighbors': range(1, 21),    # Number of neighbors to consider
    'weights': ['uniform', 'distance'],  # Weighting scheme for neighbors
    'p': [1, 2],  # Power parameter for the Minkowski distance metric
}

# Create the KNN classifier
knn = KNeighborsRegressor(n_neighbors=5)  # Set the number of neighbors (k) as 5

# Initialize the RandomizedSearchCV object
rscv_knn = RandomizedSearchCV(estimator=knn, param_distributions=param_grid, cv=5, n_jobs=-1, verbose=2, n_iter=

# Perform hyperparameter tuning
rscv_fit_knn = rscv_knn.fit(x_train_knn, y_train_knn)

# Get the best parameters and best model
best_params = rscv_fit_knn.best_params_
best_model = rscv_fit_knn.best_estimator_
```

```
print(best_params)
print(best_model)
```

```
Fitting 5 folds for each of 50 candidates, totalling 250 fits
{'weights': 'distance', 'p': 1, 'n_neighbors': 5}
KNeighborsRegressor(p=1, weights='distance')
```

In [54]:
```
# Retrain the best model on the full training set
best_model.fit(x_train_knn, y_train_knn)

# Make predictions
predict = best_model.predict(x_test_knn)
predict
```

Out[54]:
```
array([16832.38646852, 14340.79699897, 10770.59679122, 10148.3706837 ,
       11633.85758822, 10950.42518393, 11625.52565641, 18442.5005618 ,
       11799.74631531, 17736.51096854, 11358.62938979, 18037.76558621,
       18641.21332006, 11520.31589956, 11775.92232687, 11604.8265767 ,
       10784.36451466, 10803.78946032, 10857.45167647, 11989.93200368,
       17514.98832427, 17421.29830843, 18342.20504081,  9487.81008889,
       13892.36809633, 16328.07320963, 19594.66404128, 17556.55775147,
       10855.04105488, 10363.34103084, 17364.08859931, 12101.1765679 ,
       15997.78821363, 17426.98021373, 17243.55500888, 11128.86519932,
       11056.2827821 ,  9190.94402717, 17550.88191563, 11440.06874488,
       17273.23889635, 12090.92507216, 10775.40384785, 10913.15572473,
       11691.28802542, 15785.98911457, 10588.45909436, 10520.11133781,
       17786.34329124,  9886.80885008, 18172.62329914, 17784.39223733,
       10215.69875432, 16595.63225438, 11946.81776821, 11904.77837836,
       10721.5759018 , 12156.02078779, 17125.93616775, 17240.08056401,
       10627.90086541, 15049.41531205, 10838.85009915, 19586.84663556,
       18002.60133055, 11773.46706188, 11126.06081565, 17816.031042  ,
       12140.20110631, 10559.79761248, 12184.92526579, 11709.01748326,
       10423.0585267 , 19487.7167148 , 10728.4362635 , 17733.13275012,
       11849.4645602 , 10026.72935536, 18564.43131443, 10557.91670812,
       19573.84402619, 18098.54143345, 15872.17836409, 17713.00595902,
       19312.09950871, 11821.24393756, 17213.80772021, 16494.13503189,
       10943.12401155, 10972.44187893, 11257.31117906,  9380.21423682,
       18287.40705294, 11855.26495687, 17087.98343612, 14817.38970466,
       17672.25247778, 10633.89051883, 16459.36448285, 10830.49743249,
       10556.06089342, 16766.1357886 , 18212.63596097,  9319.44059896,
       17736.22785907, 10806.87245673, 19648.97317252, 11162.23428345,
       10072.86280328, 15901.66218425, 19475.18253861, 16806.7134151 ,
       14628.87794746, 11543.58002268, 15727.1134947 , 11619.9149945 ,
       17168.42397646, 11498.69029222, 10526.97338441, 11892.64929156,
       17662.80289632, 14715.84510488, 13844.80029348, 17552.40770717,
       13048.56660485,  8379.49312365, 17127.29350616, 10587.49758655,
       10543.52503359, 11327.85152962, 10578.55692441, 17735.96416145,
       10441.57592092,  8896.02076315, 11167.51167619, 13817.41012604,
       14234.54670861, 11297.61916693, 17363.24692605, 14367.55210358,
       10855.31788369, 18325.33574   , 17326.75837276, 11634.83005535,
       10338.42416168, 15828.700958  , 12227.9131942 , 10862.0874266 ,
        9337.2080592 , 17947.80975185, 15079.61340567, 11616.11138864,
       15814.98437994, 17913.97386656, 10684.33874846,  9800.64725399,
       11962.77110771, 18003.20884675, 17988.547764  , 11720.24146612,
       17138.66468588, 11931.3203356 , 10938.36995605, 17890.97082378,
       18639.11039234, 18336.22693473, 10743.68703365, 17657.9546064 ,
       17350.73922284, 10460.5907008 , 12194.34544366, 17920.28590785,
       16235.22960288, 15813.2201591 ,  9136.80912959, 12265.40274889,
        9623.89743545, 11443.68958129, 12231.14621217, 11360.02363611,
       17396.44913699, 11575.19910885, 16518.83989823, 10604.54427572,
       17817.42393417, 10995.85386628, 18235.40573838, 14576.38286083,
       18516.38052847, 15876.2635158 , 10704.73480953, 17093.28247169,
       17548.48973867, 10589.23877267, 14692.87235025,  9188.05553986,
       18615.89223684, 15784.35287963, 18082.1410501 , 10706.68120011,
       16262.85910281, 14655.26671731, 10820.07882581, 13904.1126096 ,
       10274.83053356, 10360.78875614, 10578.40049987, 17189.47562321,
       11604.93485727, 15941.82721708, 17269.65191309,  8703.86059749,
       19522.90829812, 11948.12378465,  8970.42504716, 16857.75098532,
       11390.77370275, 10695.39887116, 19588.46307567, 11327.51563911,
       11009.388139  , 10604.02964481, 10599.69920847,  9275.81923219,
       17985.69917516, 18139.8552438 , 15809.07782375, 11624.71221608,
       10462.11165567, 16956.56987905, 17419.51470853, 11395.16263175,
       17616.77591429, 12010.49736077, 17792.53436505, 12062.31066584,
       11937.33683508, 17372.24382434, 17707.21549429, 12280.96768591,
       11951.90137795, 11756.23385235, 17085.35045068, 11417.59946378,
       10359.10112997, 11909.0336313 , 10796.20775724, 18050.52036796,
       11994.62080484,  8353.09545285, 14544.02531633, 11875.01189216,
       15505.31265281, 11996.82063521, 14887.74711475, 11457.76375703,
       14636.90541435, 11382.60983886, 11427.5510853 , 11978.41763466,
       18658.95988169, 18291.44597287, 18272.80722983, 17092.81629257,
       12773.14735428, 17435.91048769, 14790.38044704, 17102.38882046,
       11410.81214537, 11662.68342905, 11049.94616079, 18078.39730161,
       17505.05900397, 16260.77460982])
```

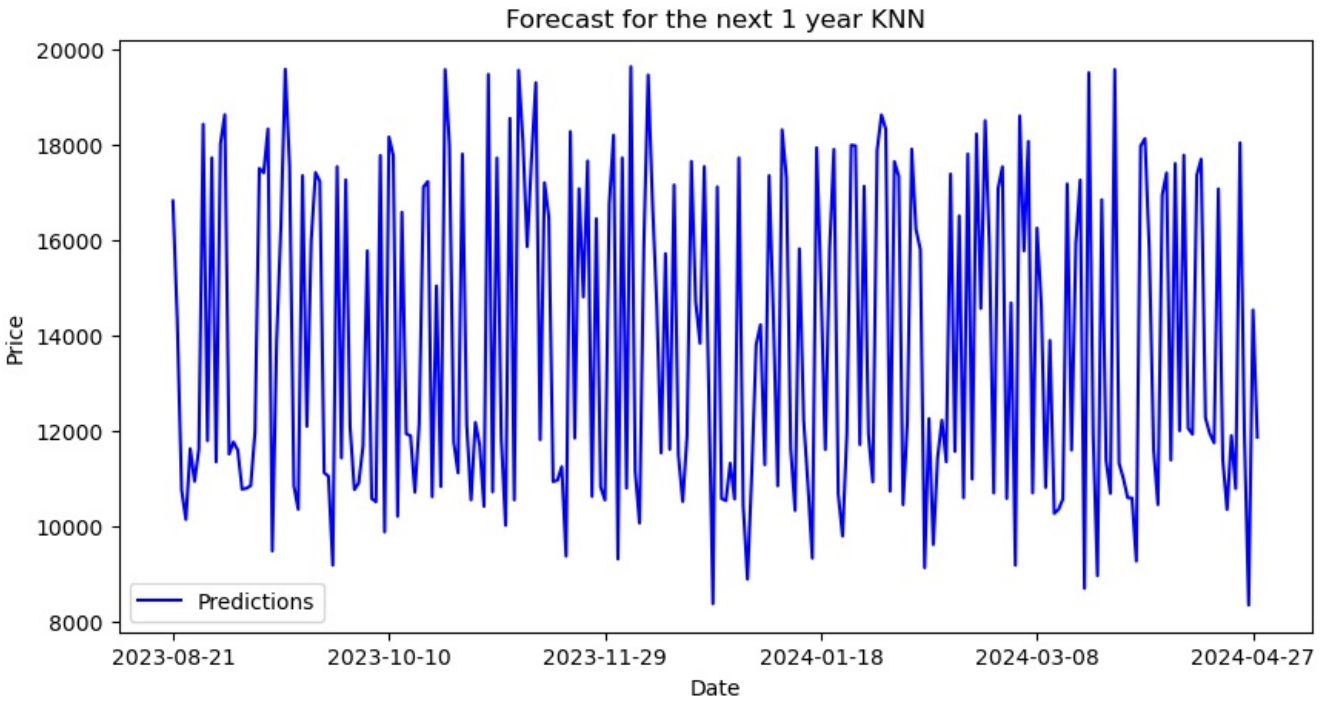
In [55]:
```
print("Mean Absolute Error:", round(metrics.mean_absolute_error(y_test_knn, predict), 4))
print("Mean Squared Error:", round(metrics.mean_squared_error(y_test_knn, predict), 4))
print("Root Mean Squared Error:", round(np.sqrt(metrics.mean_squared_error(y_test_knn, predict)), 4))
print("(R^2) Score:", round(metrics.r2_score(y_test_knn, predict), 4))
print(f'Train Score : {model.score(x_train_knn, y_train_knn) * 100:.2f}% and Test Score : {model.score(x_test_kn
errors = abs(predict - y_test_knn)
mape = 100 * (errors / y_test_knn)
accuracy = 100 - np.mean(mape)
print('Accuracy:', round(accuracy, 2), '%.')
```

```
Mean Absolute Error: 103.5042
Mean Squared Error: 25588.6683
Root Mean Squared Error: 159.9646
(R^2) Score: 0.9976
Train Score : 99.99% and Test Score : 99.99% using KNN Regressor.
Accuracy: 99.21 %.
```

In [56]:
```
predictions = pd.DataFrame({"Predictions": predict})
predictions['Date'] = pd.date_range(start=nse_data.index[-1], periods=len(predict), freq="D")
predictions.set_index("Date", inplace=True)
predictions.to_csv("Predicted-price-data_knn_NSE.csv")
#colllects future days from predicted values
oneyear_df = pd.DataFrame(predictions[:252])
oneyear_df.to_csv("one-year-predictions_knn_NSE.csv")
onemonth_df = pd.DataFrame(predictions[:21])
onemonth_df.to_csv("one-month-predictions_knn_NSE.csv")
fivedays_df = pd.DataFrame(predictions[:5])
fivedays_df.to_csv("five-days-predictions_knn_NSE.csv")
```

In [57]:
```
oneyear_df_pred = pd.read_csv("one-year-predictions_knn_NSE.csv")
oneyear_df_pred.set_index("Date", inplace=True)
buy_price = min(oneyear_df_pred["Predictions"])
sell_price = max(oneyear_df_pred["Predictions"])
oneyear_buy = oneyear_df_pred.loc[oneyear_df_pred["Predictions"] == buy_price]
oneyear_sell = oneyear_df_pred.loc[oneyear_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(oneyear_buy)
print("Sell price and date")
print(oneyear_sell)
oneyear_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 1 year KNN", color="blue")
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()
```

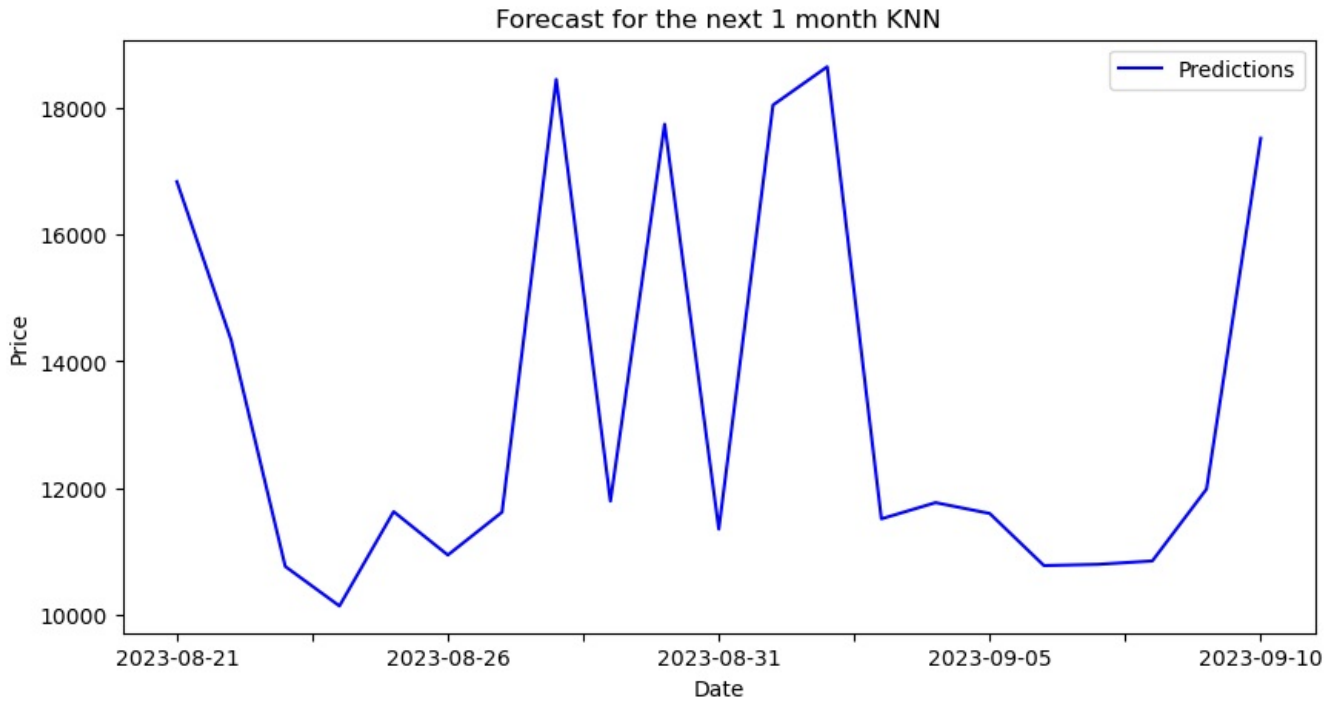
```
Buy price and date
            Predictions
Date
2024-04-26  8353.095453
Sell price and date
             Predictions
Date
2023-12-05  19648.973173
```



In [58]:
```
onemonth_df_pred = pd.read_csv("one-month-predictions_knn_NSE.csv")
onemonth_df_pred.set_index("Date", inplace=True)
```

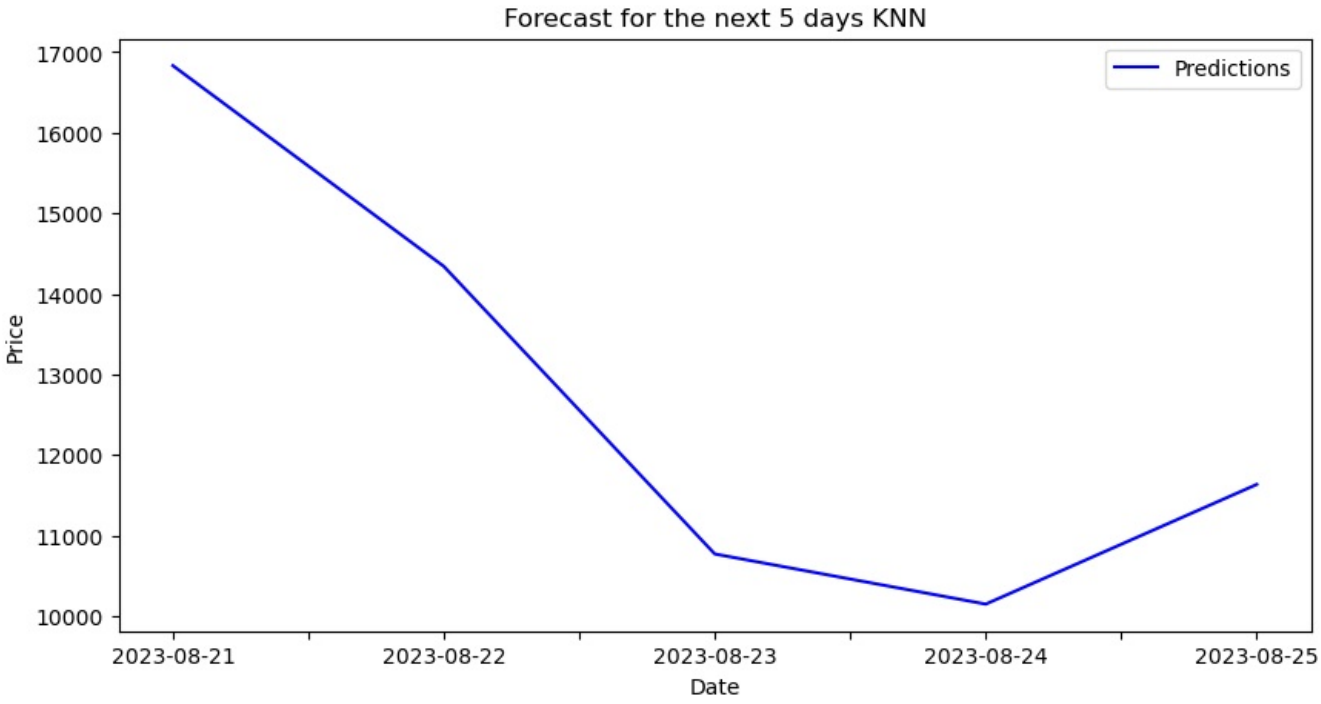
```
buy_price = min(onemonth_df_pred["Predictions"])
sell_price = max(onemonth_df_pred["Predictions"])
onemonth_buy = onemonth_df_pred.loc[onemonth_df_pred["Predictions"] == buy_price]
onemonth_sell = onemonth_df_pred.loc[onemonth_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(onemonth_buy)
print("Sell price and date")
print(onemonth_sell)
onemonth_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 1 month KNN", color="blue")
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()
```

```
Buy price and date
            Predictions
Date
2023-08-24  10148.370684
Sell price and date
            Predictions
Date
2023-09-02  18641.21332
```



In [59]:
```
fivedays_df_pred = pd.read_csv("five-days-predictions_knn_NSE.csv")
fivedays_df_pred.set_index("Date", inplace=True)
buy_price = min(fivedays_df_pred["Predictions"])
sell_price = max(fivedays_df_pred["Predictions"])
fivedays_buy = fivedays_df_pred.loc[fivedays_df_pred["Predictions"] == buy_price]
fivedays_sell = fivedays_df_pred.loc[fivedays_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(fivedays_buy)
print("Sell price and date")
print(fivedays_sell)
fivedays_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 5 days KNN", color="blue")
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()
```

```
Buy price and date
            Predictions
Date
2023-08-24  10148.370684
Sell price and date
            Predictions
Date
2023-08-21  16832.386469
```



Clustering

In [60]:
```
temp = nse_data
temp = temp.drop('Adj Close', axis=1)
x = temp.iloc[:, 0:12].values
y = nse_data.iloc[:, 4].values
```

In [61]:
```
x_train_cls, x_test_cls, y_train_cls, y_test_cls = train_test_split(x, y, test_size=0.20,  random_state=0)
```

In [62]:
```
scale = StandardScaler()
x_train_cls = scale.fit_transform(x_train_cls)
x_test_cls = scale.transform(x_test_cls)
```

In [63]:
```
# Create the SVR regressor
svr_regressor = SVR()

# Define the parameter grid to search over
param_grid = {
    'kernel': ['linear', 'poly', 'rbf'],
    'C': [0.1, 1.0, 10.0],
    'epsilon': [0.01, 0.1, 1.0]
}

# Initialize the GridSearchCV object
grid_search = GridSearchCV(estimator=svr_regressor, param_grid=param_grid, cv=3, scoring='neg_mean_squared_erro

# Perform hyperparameter tuning
grid_search.fit(x_train_cls, y_train_cls)

# Get the best hyperparameters and best model
best_params = grid_search.best_params_
best_model = grid_search.best_estimator_

print(best_params)
print(best_model)
```

```
{'C': 10.0, 'epsilon': 1.0, 'kernel': 'linear'}
SVR(C=10.0, epsilon=1.0, kernel='linear')
```

In [64]:
```
# Retrain the best model on the full training set
best_model.fit(x_train_cls, y_train_cls)

# Make predictions
```

```
predict = best_model.predict(x_test_cls)
predict
```

Out[64]:
```
array([16683.34225705, 13993.59020054, 10596.5393215 , 10113.4544746 ,
       11729.40588819, 11033.00422495, 11722.05068419, 18449.95488793,
       11731.96011227, 17792.51493974, 11305.88128921, 17979.40622406,
       18606.97765779, 11288.57493425, 11788.45774109, 11683.79597137,
       10844.22312467, 10738.15191814, 10894.57667676, 12213.78884211,
       17537.62382954, 17255.85573397, 18355.34652297,  9336.05813955,
       13913.77204031, 16242.73033881, 19663.91364994, 17583.79747362,
       10858.62301518, 10348.33014423, 17308.68401497, 12149.66688113,
       15982.13564587, 17339.43641006, 17330.82474426, 11065.33414584,
       11184.06104731,  9177.26329322, 17536.0961078 , 11441.49171531,
       17222.57344751, 12281.71022293, 10860.53953985, 10885.52662788,
       11902.04380589, 15818.27224399, 10567.4769437 , 10582.89221146,
       17717.26700007, 10015.63692474, 18290.28260563, 17684.15572296,
       10139.63219315, 16641.25889727, 11999.00836167, 11907.94150963,
       10689.32382951, 12267.54681451, 16959.30750365, 17245.33430404,
       10589.99708303, 15164.19268878, 10854.65889074, 19620.35154205,
       17922.23107716, 11851.38104153, 11060.12144547, 17735.52196111,
       12172.82315897, 10515.39479708, 12160.21509439, 11738.03911456,
       10394.00697114, 19341.94787037, 10616.31951466, 17806.56576665,
       11734.08241607,  9890.36435458, 18754.82418575, 10563.77681356,
       19644.30491476, 18127.14951221, 16049.87125897, 17581.00191678,
       19375.84131784, 12017.12532495, 17107.11648655, 16353.04746216,
       10963.37724761, 10979.6082763 , 11192.41133012,  9280.92687247,
       18309.70406988, 11793.25441295, 16730.23146478, 14923.80568956,
       17691.31259306, 10592.67585439, 16593.65753959, 10732.05336636,
       10581.79203237, 16512.90564787, 18200.43251011,  9286.01568096,
       17672.28631276, 10798.0602372 , 19688.78265052, 11187.55578893,
       10224.46689799, 15795.22021082, 19445.48106976, 16847.91454275,
       14656.15911139, 11569.72413711, 15695.83373366, 11493.22316752,
       16975.46183884, 11517.88020679, 10504.70584648, 12078.32697234,
       17552.75017956, 14575.26966155, 14022.78924888, 17627.78594721,
       13079.65222453,  8468.82618225, 17113.99972261, 10984.14869483,
       10541.01676942, 11310.67819691, 10560.48661809, 17720.19197423,
       10453.68808629,  8856.12374392, 11078.84771347, 13869.36760414,
       14080.2382661 , 11304.41120908, 17204.59634395, 14338.18424682,
       10860.33548141, 18379.22794642, 17090.73378147, 11599.78673345,
       10212.24010129, 15735.52015229, 12360.38995098, 10989.10085907,
        9288.11549248, 17945.32382996, 15199.27304176, 11588.43656815,
       15795.76687071, 17624.45270787, 10688.31800061,  9816.70529938,
       12030.76555135, 17945.74495373, 17987.54463971, 11670.13938222,
       16974.66596207, 11851.41510328, 10896.15585596, 17957.74621903,
       18737.87606797, 18479.3897701 , 10731.56101294, 17644.68889674,
       17522.65476421, 10406.45399726, 12234.58734351, 17758.1135606 ,
       16174.50418931, 15839.88816043,  8979.96825638, 12254.48493013,
        9996.53599964, 11406.93800752, 12292.13496355, 11337.35120307,
       17209.46316727, 11335.71945932, 15388.63292804, 10499.2684949 ,
       18016.02759608, 10866.17851392, 18202.11917201, 14426.74799138,
       18538.74757548, 15940.10482222, 10626.12948175, 17087.22364542,
       17467.92477408, 10294.60702423, 14572.85528814,  9046.09105016,
       18488.25195908, 15717.10935482, 18157.02947297, 10670.73233181,
       16238.62424823, 14625.2030475 , 10813.83382811, 13235.58183986,
       10089.89928535, 10244.78806827, 10458.49521697, 17063.34443606,
       11559.82716961, 15438.2868007 , 17109.56704997,  9350.9543733 ,
       19331.40102214, 11857.70248843,  8095.99161835, 16623.1321893 ,
       11299.45235608, 10543.36511474, 19551.8078639 , 11299.27664235,
       11040.5926189 , 10560.12462905, 10564.45897496,  9162.1083015 ,
       17841.79720023, 18303.46284064, 15851.88391068, 11578.92933054,
       10432.16855829, 16887.42831009, 17147.98023549, 11517.79433841,
       17679.98403764, 11998.45656231, 17891.65809227, 12044.63658883,
       11920.66045183, 17323.16838548, 17652.6551177 , 12260.06884876,
       11928.25283669, 11655.65514948, 16995.85068661, 11807.80403266,
       10362.39307621, 11874.7781966 , 10818.74800378, 18161.61721325,
       11969.33064687,  8579.39928068, 14688.18092984, 11967.15362243,
       15631.41131727, 12047.38449943, 15215.97502301, 11585.62899195,
       14396.12841141, 11354.45965724, 11317.53738543, 11947.26035892,
       18665.75952261, 18267.02760961, 18288.25914431, 17066.42637962,
       12773.47821952, 17352.29737105, 14843.95068474, 16967.56013728,
       11313.26706403, 11634.92879956, 11062.72271683, 18174.02713304,
       17478.9791005 , 16206.54552667])
```

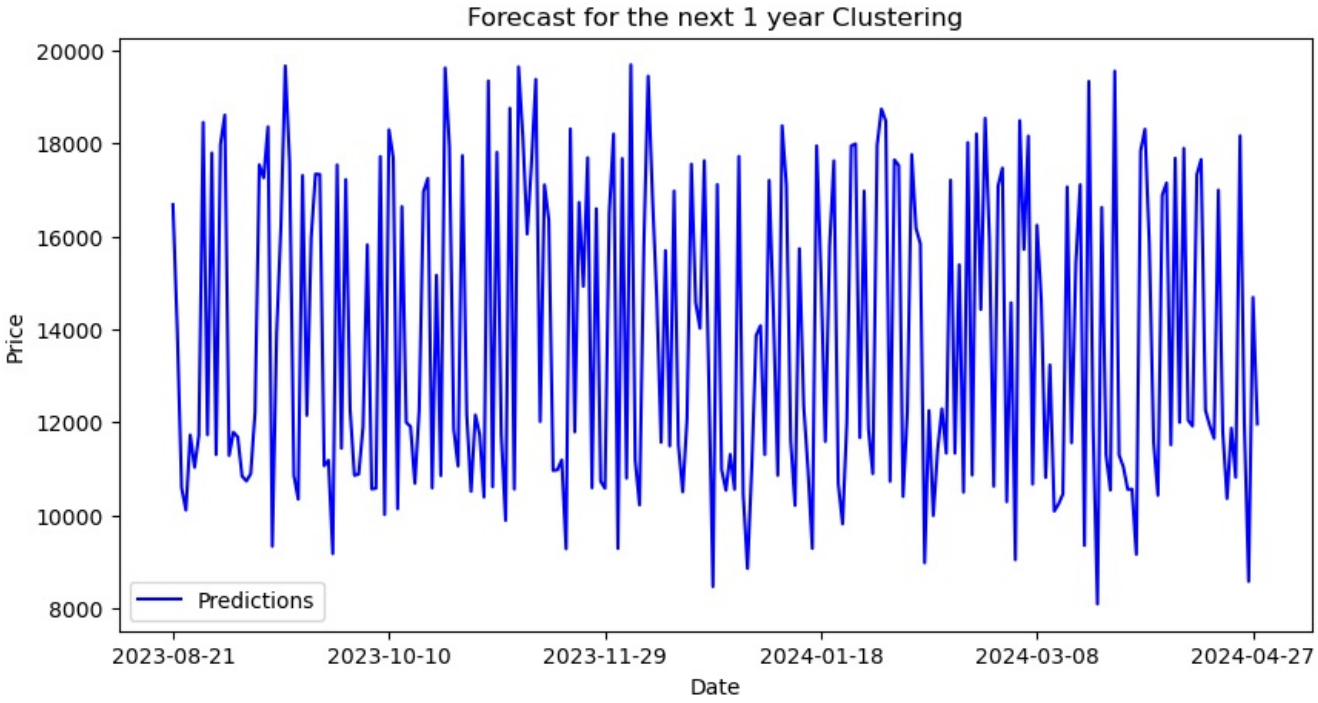
In [65]:
```
print("Mean Absolute Error:", round(metrics.mean_absolute_error(y_test_cls, predict), 4))
print("Mean Squared Error:", round(metrics.mean_squared_error(y_test_cls, predict), 4))
print("Root Mean Squared Error:", round(np.sqrt(metrics.mean_squared_error(y_test_cls, predict)), 4))
print("(R^2) Score:", round(metrics.r2_score(y_test_cls, predict), 4))
print(f'Train Score : {model.score(x_train_cls, y_train_cls) * 100:.2f}% and Test Score : {model.score(x_test_c
errors = abs(predict - y_test_cls)
mape = 100 * (errors / y_test_cls)
accuracy = 100 - np.mean(mape)
print('Accuracy:', round(accuracy, 2), '%.')
```

```
Mean Absolute Error: 21.6668
Mean Squared Error: 1063.9454
Root Mean Squared Error: 32.6182
(R^2) Score: 0.9999
Train Score : 99.99% and Test Score : 99.99% using SVR Clustering Regressor.
Accuracy: 99.83 %.
```

In [66]:
```
predictions = pd.DataFrame({"Predictions": predict})
predictions['Date'] = pd.date_range(start=nse_data.index[-1], periods=len(predict), freq="D")
predictions.set_index("Date", inplace=True)
predictions.to_csv("Predicted-price-data_cls_NSE.csv")
#colllects future days from predicted values
oneyear_df = pd.DataFrame(predictions[:252])
oneyear_df.to_csv("one-year-predictions_cls_NSE.csv")
onemonth_df = pd.DataFrame(predictions[:21])
onemonth_df.to_csv("one-month-predictions_cls_NSE.csv")
fivedays_df = pd.DataFrame(predictions[:5])
fivedays_df.to_csv("five-days-predictions_cls_NSE.csv")
```

In [67]:
```
oneyear_df_pred = pd.read_csv("one-year-predictions_cls_NSE.csv")
oneyear_df_pred.set_index("Date", inplace=True)
buy_price = min(oneyear_df_pred["Predictions"])
sell_price = max(oneyear_df_pred["Predictions"])
oneyear_buy = oneyear_df_pred.loc[oneyear_df_pred["Predictions"] == buy_price]
oneyear_sell = oneyear_df_pred.loc[oneyear_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(oneyear_buy)
print("Sell price and date")
print(oneyear_sell)
oneyear_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 1 year Clustering", color="blu
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()
```

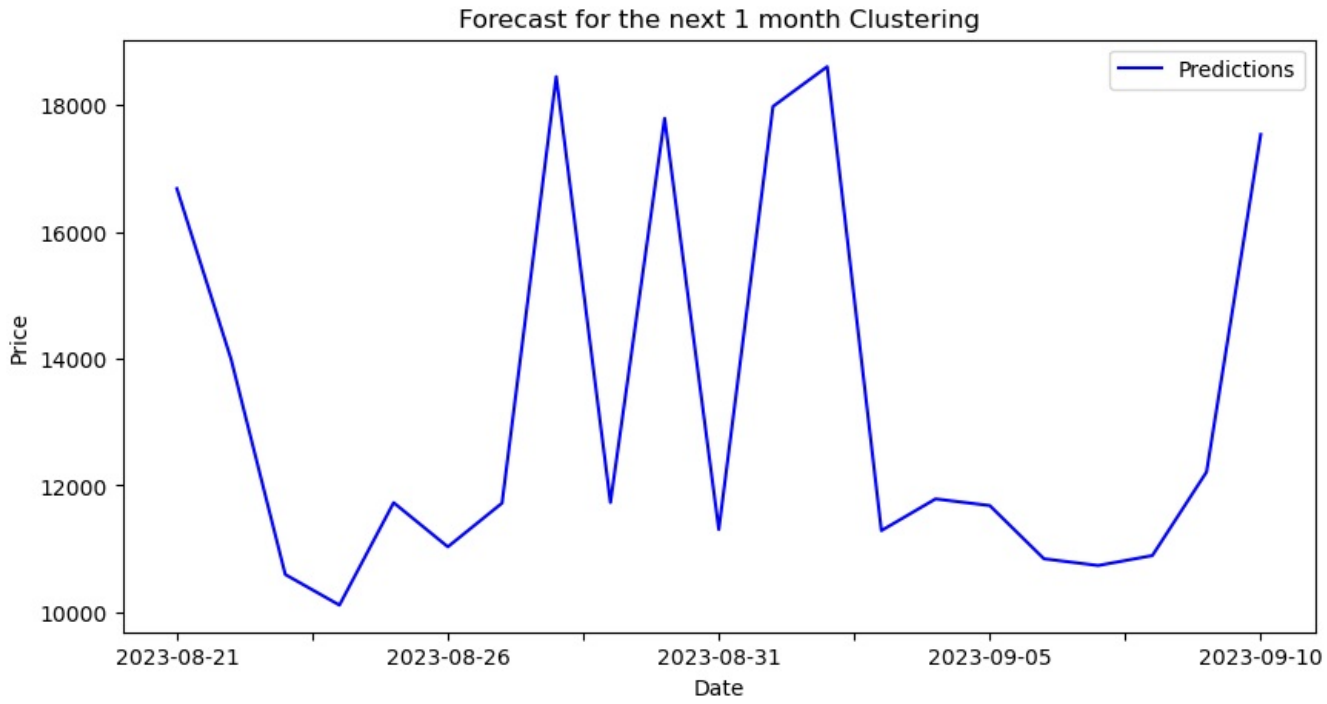
```
Buy price and date
            Predictions
Date
2024-03-22  8095.991618
Sell price and date
             Predictions
Date
2023-12-05  19688.782651
```



In [68]:
```
onemonth_df_pred = pd.read_csv("one-month-predictions_cls_NSE.csv")
onemonth_df_pred.set_index("Date", inplace=True)
buy_price = min(onemonth_df_pred["Predictions"])
sell_price = max(onemonth_df_pred["Predictions"])
onemonth_buy = onemonth_df_pred.loc[onemonth_df_pred["Predictions"] == buy_price]
onemonth_sell = onemonth_df_pred.loc[onemonth_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(onemonth_buy)
print("Sell price and date")
print(onemonth_sell)
onemonth_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 1 month Clustering", color="
plt.xlabel("Date")
```

```
plt.ylabel("Price")
plt.legend()
plt.show()
```

```
Buy price and date
             Predictions
Date
2023-08-24  10113.454475
Sell price and date
             Predictions
Date
2023-09-02  18606.977658
```



In [69]:
```
fivedays_df_pred = pd.read_csv("five-days-predictions_cls_NSE.csv")
fivedays_df_pred.set_index("Date", inplace=True)
buy_price = min(fivedays_df_pred["Predictions"])
sell_price = max(fivedays_df_pred["Predictions"])
fivedays_buy = fivedays_df_pred.loc[fivedays_df_pred["Predictions"] == buy_price]
fivedays_sell = fivedays_df_pred.loc[fivedays_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(fivedays_buy)
print("Sell price and date")
print(fivedays_sell)
fivedays_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 5 days Clustering", color="b
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()
```

```
Buy price and date
             Predictions
Date
2023-08-24  10113.454475
Sell price and date
             Predictions
Date
2023-08-21  16683.342257
```